

Sapientia Erdélyi Magyar Tudományegyetem

Marosvásárhelyi Kar

Bus4U

TDK Dolgozat

Témavezető:

Dr. Szántó Zoltán

Egyetemi adjunktus

Hallgatók:

Portik Szabolcs

Zsigmond Attila

Számítástechnika IV.

2025

Tartalomjegyzék

1. Bevezető	2
2. Célkitűzések	4
3. Szakirodalmi tanulmány	6
3.1. Létező megoldások	10
3.1.1. Moovit	10
3.1.2. Budapest Go	11
4. A rendszer specifikációi	13
4.1. Felhasználói követelmények	13
4.1.1. Felhasználói felület	13
4.1.2. Adminisztrátori felület	14
4.2. Rendszerkövetelmények	16
4.2.1. Funkcionális követelmények	16
4.2.2. Nem funkcionális követelmények	18
5. A rendszer architektúrája	19
5.1. Frontend	20
5.2. Backend	21
5.3. Kommunikáció	22
5.4. Felhasznált technológiák	26
5.4.1. Frontend	26

5.4.2. Backend	28
5.5. Fejlesztői eszközök	30
5.6. Kitelepítés	31
5.6.1. Azure infrastruktúra áttekintése	31
5.6.2. Domain konfiguráció	31
5.6.3. Adatbázis telepítése és konfigurációja	32
5.6.4. Szerver telepítése	32
6. Adatkezelés	34
6.1. API végpontok részletes bemutatása	34
6.2. Adatbázis kapcsolat	37
6.2.1. Adatbázis táblák	37
6.2.2. Objektum-relációs leképzés (ORM)	40
7. A felhasználói felület tervezése	41
7.1. Figma terv	41
8. Gyakorlati megvalósítás	44
8.1. Felhasználói weboldal és alkalmazás	44
8.1.1. Főoldal	44
8.1.2. Jegyek	47
8.1.3. Menetrend	48
8.1.4. Megállók és élő térkép	49
8.2. Adminisztrátori weboldal	50
8.2.1. Kezdőoldal	50
8.2.2. Megállók	51
8.2.3. Útvonalak	51
8.2.4. Menetrend	52
8.2.5. Buszok és alkalmazottak	53
8.3. POS terminál alkalmazása	55

8.3.1. GPS oldal	55
8.3.2. Jegyszkennelő oldal	56
9. Összefoglalás	57
9.1. További fejlesztési lehetőségek	58
Irodalomjegyzék	59

Kivonat

Napjaink felgyorsult világában kevesen használják a tömegközlekedést, mert az nem elég kényelmes, nem elég gyors vagy éppen nem úgy működik, ahogy azt elvárnánk tőle. A gyakori problémák közé tartozik, hogy a buszok nem tartják a menetrendet vagy a jegyek megvásárlása nehézkes. Sok helyen a menetrend nincs kifüggesztve a megállókban és az interneten sem elérhető. Az is gyakran megesik hogy a buszok nagyon zsúfoltak és erről az utazni vágyók csak akkor vesznek tudomást, amikor már a megállóban vannak és nem áll rendelkezésre más utazási lehetőség.

A fent említett problémákra kínál megoldást a Bus4U online platform, amely egy mobilalkalmazásból, egy felhasználói weboldalból és egy adminisztrátori weboldalból tevődik össze. A végfelhasználók számára elérhető funkciók közé tartozik a buszjáratok keresése a kiindulópont, a célállomás és az időpont megadásával, a járatokra szóló előzetes online jegyvásárlás, illetve a járatok indulási időpontjainak megtekintése városokra és megállókra lebontva. Ezen kívül megtekinthetők a buszmegállók a térképen, városok és útvonalak szerint szűrve, valamint valós időben követhetőek az autóbuszok pillanatnyi helyzetei is.

Az alkalmazás másik fontos része az adminisztrációs felület, amelyet a busztársaságok használhatnak. Ezen a felületen lehetőség van a menetrendek kezelésére, új útvonalak és megállók felvitelére, valamint a meglévő adatok frissítésére. Továbbá itt kezelhetők az alkalmazottak információi, beleértve a sofőrök és egyéb személyzet adatait, valamint az autóbuszok műszaki és üzemeltetési adatai. Ez az adminisztrációs rendszer biztosítja a busztársaságok adatainak karbantartását, így azok könnyedén frissíthetik szolgáltatásaikat és biztosíthatják a pontosságot.

A rendszer különböző szolgáltatásainak biztosításához, mint például a jegykezelés, szükség van egy másik alkalmazásra is, amely a buszokon található POS terminálon fut. Ez biztosítja a jegyek érvényesítését, illetve az autóbuszok élő információinak megosztását is a központi szerverrel.

Továbbfejlesztési lehetőségek között szerepelnek a bérletek és különböző kedvezmények létrehozása, valamint a buszok telítettségének követése valós időben. A busztársaságok szempontjából fontos lehet még a sofőrök munkaidejének követése és az automatizált könyvelés bevezetése is.

Kulcsszavak: tömegközlekedés, buszjárat, digitalizálás

1. fejezet

Bevezető

A tömegközlekedési rendszerek sajnos nem túl népszerűek az utazók körében, egyrészt az autók kényelme miatt, másrészt a tömegközlekedésben jelenlévő problémák miatt. Ezen problémák közé tartoznak a buszok pontatlansága, a zsúfoltság és a hiányos információk. Sok helyen a buszok nem is rendelkeznek menetrenddel, vagy ha igen, azok nem pontosak. Digitalizáció szempontjából is rengeteg hiányosság van a tömegközlekedésben, így ez az egyik legelavultabb szolgáltatások egyike.

Jelenleg már ott tartunk, hogy az emberek többsége okostelefonnal rendelkezik, különböző alkalmazásokat használ és böngészik az interneten. Ezt rengeteg vállalkozás ki is használja, hogy felgyorsítsa és egyszerűsítse a szolgáltatásait. A tömegközlekedésben is egyre több vállalkozás próbálja megoldani a problémákat, de még mindig sok a hiányosság. Ezeket belátva nem is csoda, hogy az emberek inkább az autózást választják, mint a tömegközlekedést.

Egy jól megtervezett és kivitelezett tömegközlekedési rendszer sokat segítené úgy az embereknek, mint a környezetnek. Egyre több problémát okoz ugyanis a globális felmelegedés, amelynek egyik okozója az autók károsanyag kibocsátása. A tömegközlekedés egyik legnagyobb előnye, hogy sok ember szállításával kevesebb szennyező anyag kerül a levegőbe, mint az autózás esetén. Ezen kívül a tömegközlekedés sokkal kisebb területet foglal el, mint az autózás, így a városokban is sokkal kevesebb helyet foglal el a parkolás. Egyes városokban már most is problémát okoz a parkolóhelyek hiánya és a hatalmas dugók, amelyeket az autók okoznak. Ezt mindenki tapasztalja nap mint nap, de sajnos így is kevesen választják a tömegközlekedést.

Azért választottuk ezt a témát, mert mindketten megtapasztaltuk a tömegközlekedés és a városi

forgalom problémáit a mindennapi ingázás során és úgy gondoltuk, hogy egy digitalizált rendszer sokat segíthetne a hozzánk hasonló ingázóknak és az alkalmi utazóknak is. Ha az emberek interneten tudnának tájékozódni a buszokról és a forgalomról, vásárolhatnak jegyet, és láthatnák a buszok valós idejű pozícióját, valószínűleg sokkal többen választanák a tömegközlekedést az autózás helyett.

A dolgozat célja egy olyan rendszer bemutatása, amely a digitalizáció által megoldja a tömegközlekedésben jelen lévő problémákat, ezáltal kényelmesebbé és egyszerűbbé téve a tömegközlekedést. Ez a rendszer nem csak az utasoknak nyújthat segítséget, hanem a busztársaságoknak is, mert ezt használva egyszerűbben tudják intézni online az ügyeiket és jobban át tudják tekinteni a vállalatukat.

A szóban forgó rendszer neve Bus4U, amely egy végfelhasználóknak szánt platformból és egy adminisztrátori felületből tevődik össze. A felhasználói felület főbb funkciói közé tartozik a buszjáratok keresése, a járatokra szóló előzetes online jegyvásárlás, illetve a járatok indulási időpontjainak megtekintése városokra és megállókra lebontva. Ezen kívül megtekinthetők a buszmegállók a térképen, városok és útvonalak szerint szűrve, valamint valós időben követhetők az autóbuszok pillanatnyi helyzetei is.

Az adminisztrációs felületen lehetőség nyílik elsősorban a menetrendek és a megállók kezelésére, valamint az útvonalak feltöltésére. A busztársaság adminisztrátora itt tudja kezelni az alkalmazottak információit, és az autóbuszok különböző adatait is, mint az időszakos műszaki vizsga és a biztosítás. Ugyanitt található egy statisztikai felület is, ahol a felhasználók számát, az eladott jegyek számát és az ezekből generált bevételt lehet megtekinteni havi, éves és mindenkori bontásban.

A dolgozatban bemutatásra kerül a rendszer célja, követelményei, architektúrája és működése. A dolgozat végén pedig összegzésre kerülnek a legfontosabb információk és a jövőbeli terveink.

2. fejezet

Célkitűzések

Fő célunk egy olyan rendszer kialakítása, amely egyaránt hasznos a tömegközlekedést választó embereknek, illetve a busztársaságoknak. A teljes rendszer működése érdekében a következő célokat kell teljesíteni a felhasználók kiszolgálásának szempontjából:

- könnyen használható weboldal, illetve mobilalkalmazás elkészítése, amely gyors és egyszerű hozzáférést biztosít a tömegközlekedési információkhoz
- útvonaltervezés és jegyvásárlás: a felhasználók megadhatják a kiindulópontjukat, a célállomásukat és az időpontot, majd a rendszer megmutatja a lehetséges járatokat és azok árait, valamint lehetőséget biztosít a jegyvásárlásra
- menetrendek megtekintésének lehetősége: indulási időpontok megjelenítése, megállók és járatok szerint
- buszok útvonalának és megállóinak megtekintése: az útvonalak és megállók térképen való megjelenítése, és szűrési lehetőség biztosítása városok és útvonalak szerint
- buszok valós idejű követése: folyamatosan frissülő adatok megjelenítése a buszok pillanatnyi helyzetéről a térképen

A busztársaságok szempontjából további funkciókra is szükség van, amelyek biztosítják az adminisztrátorok számára a cég menedzseléséhez szükséges eszközöket. Ehhez a következő célok elérésére van szükség:

- egy könnyen használható adminisztrációs felület létrehozása, amely csak a busztársaságok adminisztrátorai számára elérhető
- menetrendek és megállók kezelése: menetrendek és megállók hozzáadása, módosítása és törlése
- útvonalak feltöltése: útvonalak hozzáadása, módosítása, megállók hozzárendelése és törlése
- jegyárak beállítása: útvonalakhoz, illetve útszakaszokhoz
- alkalmazottak és buszok adatainak kezelése: alkalmazottak és buszok hozzáadása, módosítása és törlése
- statisztikai adatok megtekintése: regisztrált felhasználók száma, eladott jegyek száma és bevétel, többféle bontásban
- POS terminálon futó alkalmazás elkészítése és buszokra való kihelyezése, amely biztosítja a jegyvásárlást, a jegyek érvényesítését és valós időben figyeli a busz földrajzi pozícióját

3. fejezet

Szakirodalmi tanulmány

A dolgozat témájának megértéséhez először meg kell vizsgálni néhány alapfogalmat, amelyek a dolgozatban szereplő rendszerekhez kapcsolódnak. Itt bemutatásra kerülnek a klasszikus tömegközlekedési rendszerek és a modernebb Intelligens Közlekedési Rendszerek (ITS)¹, valamint a jelenleg elérhető megoldások, mint a Moovit és a Budapest Go alkalmazások, amelyek a városi közlekedés digitalizálására törekednek. Mindezek előtt azonban érdemes megvizsgálni a tömegközlekedés használatának pozitív hatásait, hogy megértsük, miért is fontos ezzel foglalkozni.

Manapság már mindenki tapasztalja a globális felmelegedés és a környezetszennyezés hatásait, amelyek egyik fő oka a közlekedési eszközök károsanyag kibocsátása. Egy 2018-as felmérés szerint a közlekedési szektor tette ki a globális szén-dioxid kibocsátás 25%-át, ennek pedig túlnyomó része a közúti közlekedésből származik [1]. A közlekedési eszközök károsanyag kibocsátásának csökkentése érdekében egyre több városban és országban támogatják a tömegközlekedés használatát, mert ezáltal csökkenthető a környezetszennyezés és a városi dugók is. Több fejlődő országban elvégzett tanulmány alátámasztja a tömegközlekedési eszközök bevezetésével csökkenő légszennyezést. Például Tajpej első metróvonalának megnyitása 5-15%-al csökkentette a levegő CO_2 tartalmát [1].

A tömegközlekedés nem csak a környezetvédelem szempontjából lényeges, hanem a városi forgalom csökkentésében is fontos szerepet játszik. A tömegközlekedés használata csökkentheti a városi dugókat azáltal, hogy az emberek egy járműben utaznak sok személyautó helyett, így kevesebb lesz a járművekkel elfoglalt hely mérete. Egyelőre nem sokan választják a tömegközlekedést, a kisebb

¹Intelligent Transport System

kényelem miatt, de megfelelő fejlesztésekkel vonzóbbá lehet tenni azt, így csökkenthető lehet a városi forgalom és a környezetszennyezés is [2].

A városi tömegközlekedés javításával kapcsolatban már a 70-es években is felmerült az igény a buszok automatizált kezelésére, amely akkor a megállóban való torlódást szerette volna megoldani. Ekkor használták először a Busz Flotta Kezelés (BFM)² kifejezést a buszok kezelésére. Később ezt a menetrendek kezelésére is alkalmazták, amely a buszok forgalmát és a járatok menetrendjét is magában foglalta. Napjainkban további feladatokkal egészül ki a tömegközlekedés menedzsmentje, amelyek úgynevezett Intelligens Közlekedési Rendszereket (ITS) hoznak létre. Ide tartozik a baleset-megelőzés, a buszok pozícióinak követése, az energiakezelés és a fenntarthatóság is. Egyes alrendszernek saját megnevezéseik vannak, mint például a Valós Idejű Információ (RTI)³, amely az utasokat tájékoztatja valós időben a közlekedés állapotáról és az esetleges változásokról, vagy az Automatikuss Járű Helyzet (AVL)⁴ rendszer, amely a közlekedési eszközök földrajzi pozícióit követi és ezek alapján végez számításokat. A klasszikus BFM-et kibővítve az ITS-ben található elemekkel egy olyan összetett busz kezelő rendszert kapunk, amely az alap szolgáltatások mellett, lehetővé teszi a buszok helyzeteinek valós idejű követését és az utasok számára a valós idejű információk megjelenítését, és amelynek együttes megnevezése a Dynamic Bus Fleet Management [3, 4].

Az ITS rendszerek általában a következő fő komponensekből állnak [4]:

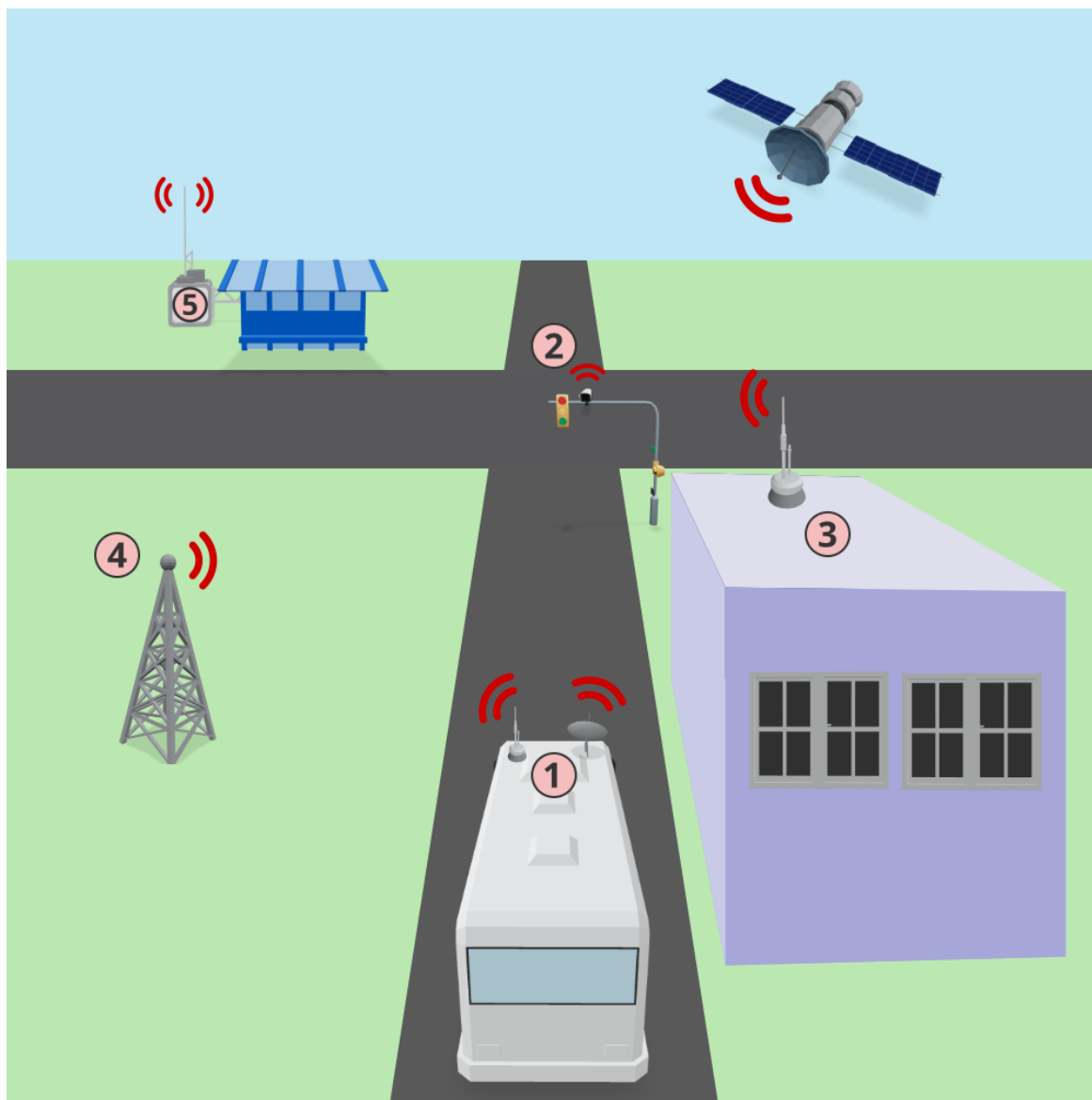
1. Járűre szerelt eszközök, mint a GPS, a kamera, a szenzorok és a kommunikációs eszközök.
2. Útmenti eszközök, mint a forgalomfigyelő kamerák és a jeladók
3. Vezérűközpont, amely a járűvek és az útmenti eszközök adatait dolgozza fel valós időben és végez számításokat ezek alapján.
4. Kommunikációs hálózat, amely a járűvek, az útmenti eszközök és a vezérűközpont között biztosítja az információ áramlását.
5. Megállóban felszerelt információ jelző eszközök, amelyek az utasokat tájékoztatják a járatok állapotáról és a menetrendről.

²Bus Fleet Management

³Real Time Information

⁴Automatic Vehicle Location

Az ITS rendszert és a benne lévő komponenseket szemlélteti a 3.1 ábra, ahol az egyes elemek számozása megegyezik az előbbi felsorolásban használt számozással.



3.1. ábra. Intelligens Közlekedési Rendszer

A buszok haladásának követésére többféle módszer is létezik, amelyek közül a legelterjedtebb a GPS alapú rendszerek használata, amelyben a buszokon levő GPS modulok határozzák meg a tartózkodási helyet és valamilyen más vezeték nélküli kommunikációs csatornán keresztül továbbítódik az adat a központi szerver felé. Sajnos a GPS működését számos tényező befolyásolhatja, mint az

időjárás, vagy a műholdjel gyenge minősége. Erre jelenthet megoldást egy új technológia, amely Bluetooth Low Energy (BLE) alapú jeladókat használ, amely a GPS-hez képest pontosabb helymeghatározást tesz lehetővé, de a hatótávolsága korlátozott. Ennek lényege hogy a buszokon csak egy kis fogyasztású Bluetooth jeladó működik, amely a megállóban lévő vevőkkel kommunikál, és ezek a vevők továbbítják az adatokat a központi szerverre. Ezzel a módszerrel a buszok helyzete pontosabban meghatározható, mint a GPS alapú rendszerekben, de a hatótávolság miatt csak a megállóban lehet használni. További nehézséget jelenthet, hogy minden megállóban fel kell szerelni egy jelfeldolgozó egységet, amelyhez áramforrás is szükséges. Nagyvárosokban ez még megoldható lenne, viszont távolsági buszjáratok esetében, ahol a megállók között nagy a távolság és nincs áramforrás minden vidéki megállóban, ott a GPS-es megoldás a legoptimálisabb [5].

Ilyen rendszerrel rendelkezik néhány nagyváros tömegközlekedése is, mint például a Londoni iBus, amely GPS alapú helymeghatározással és GPRS alapú adatátvitellel működik. Az iBus lehetővé teszi több mint 8000 busz valós idejű követését 30 másodperces frissítési rátával, amelyek adataiból kinyert információkat a buszokon található kijelzőkön és a megállóban is megjeleníti [6].

A jelenleg elérhető rendszereken még mindig lehetne fejleszteni, mert a más területek technológiai fejlődéséhez viszonyítva még mindig le van maradva a tömegközlekedés. Nagyobb a baj a kisvárosokban, ahol csak kis mértékben vagy egyáltalán nincs digitalizálva a rendszer. Ennek a helyzetnek a javításához szükség lenne egy olyan rendszerre, amely valós idejű információkat szolgáltat a járatokról, lehetővé teszi az online jegyvásárlást és az útvonaltervezést. Ehhez a buszok, vonatok és metrók különböző eszközökkel való felszerelésére van szükség, mint a jegyszkennelő automata, a kártyás fizetést biztosító POS terminál vagy a GPS modul. Egy ilyen rendszerrel való interakcióba lépéshez szükség van egy felhasználói felületre is, amely lehetővé teszi az utasok számára a különböző szolgáltatások elérését: az útvonaltervezést, a valós idejű forgalmi adatok megtekintését és a jegyvásárlást [7]. Erre a célra már léteznek megoldások, de ezek nem fedik le teljesen a busztársaságok igényeit és még messze állunk attól, hogy ezt minden város és busztársaság használni tudja.

3.1. Létező megoldások

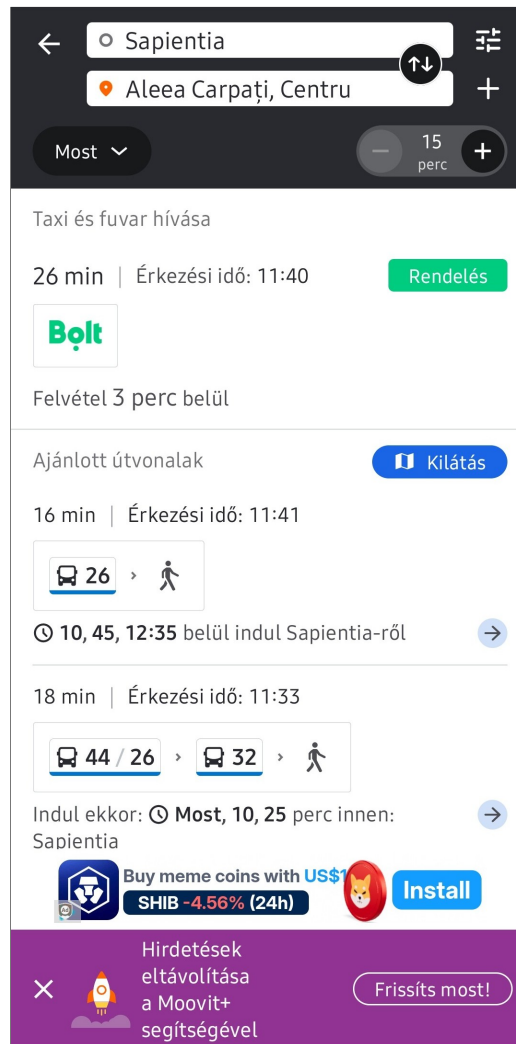
Kutatásunk során többek között rátaláltunk a Moovit és a Budapest Go nevű alkalmazásokra, amelyek két különböző megközelítést alkalmaznak a városi közlekedés digitalizálására, de céljaink hasonlóak: hogy a rendszer egyetlen platformon biztosítsa a tömegközlekedéshez kapcsolódó különféle szolgáltatásokat, beleértve a jegyvásárlást, útvonaltervezést és a valós idejű járatinformációk megjelenítését.

3.1.1. Moovit

A Moovit egy népszerű alkalmazás, amely a világ számos országában és városában működik, így egy szinte mindenhol elérhető szolgáltatást nyújt, viszonylag sok funkcióval. Az alkalmazás lehetővé teszi az utasok számára, hogy megtervezzék az útvonalukat, figyelemmel kísérik a járatokat és megvásárolják a jegyeket⁵. A közlekedési információkat az utasok okostelefonjaikon futó alkalmazások biztosítják, amelyek időnként beküldik a GPS pozícióikat a Moovit szervereire. Ezenkívül a felhasználók manuálisan is megadhatnak információkat, például a járatok késéséről vagy pillanatnyi telítettségéről. A rendszer ezeket az adatokat dolgozza fel és jeleníti meg a többi felhasználó számára, amennyiben ezek elegendő mennyiségben rendelkezésre állnak. Ezzel csak az a probléma hogy kevés országban éri el a felhasználók száma az elvárt szintet, így ezek a funkciók nem mindenhol érhetőek el [7].

Sajnos a kevés felhasználtót számláló országok közé tartozik a miénk is, így csak a hivatalosan megadott adatokat jeleníti meg a Moovit, a felhasználóktól érkezőket nem, emiatt nálunk eléggé rövid az alkalmazás által nyújtott szolgáltatások listája. Marosvásárhelyen például csak a városi buszok menetrendje és útvonala érhető el, de ehhez nem társul élő forgalmi információ és jegyvásárlási lehetőség sem. A 3.2 ábrán látható egy képernyőkép a Moovit alkalmazásról.

⁵Moovit: <https://moovit.com>



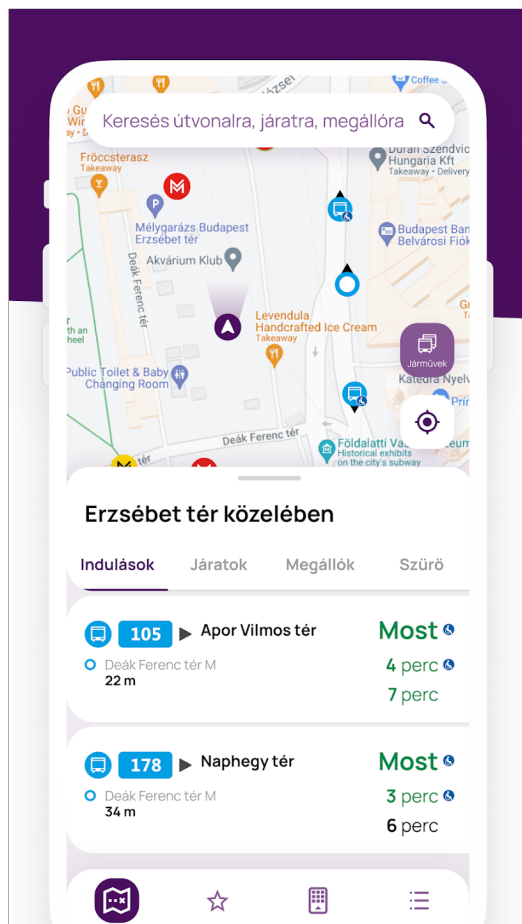
3.2. ábra. Moovit alkalmazás

3.1.2. Budapest Go

A másik alkalmazás a Budapest Go, amely egy általános közlekedési rendszer és a városi közlekedés minden elemét lefedi, így nem csak a buszokkal való közlekedésre használható, hanem majdnem az összes tömegközlekedési eszközzel a metrótól, a villamoson át, a hajóig, sőt az egyéni közlekedés megkönnyítése végett a gyalogos- és biciklis útvonalak is megtalálhatóak benne. Ez a megoldás talán közelebb áll a mi elképzeléseinkhez a funkcionalitás szempontjából, mert ugyanúgy biztosítja a jegyvásárlás lehetőségét, az útvonaltervezést és a valós idejű járatinformációkat, mint a Moovit, de a Budapest Go esetében ezek a funkciók sokkal részletesebbek és szélesebb körűek, emellett pedig

rengeteg egyéb funkciót is biztosít. A 3.3 ábrán látható a Budapest Go alkalmazás kezdőképernyője.

Egy lényeges probléma a Budapest Go-val az, hogy nem használható fel univerzálisan, mert ez a Budapesti Közlekedési Központ (BKK) saját rendszere, ezáltal pedig Budapestre van korlátozva és csak a helyi utasok tudják kihasználni az általa biztosított szolgáltatásokat⁶. Ezzel szemben mi a nyitottságra szeretnénk fektetni a hangsúlyt és egy olyan rendszert hoznánk létre, amelyet bármely busztársaság tudna használni, helytől függetlenül.



3.3. ábra. Budapest Go alkalmazás ⁷

⁶BKK x Supercharge: All-in-one mobility solution: <https://supercharge.io/work/details/budapest-go>

⁷Budapest Go - Google Play Áruház: <https://play.google.com/store/apps/details?id=hu.webvalto.bkkfutar>

4. fejezet

A rendszer specifikációi

4.1. Felhasználói követelmények

A felhasználói követelmények azokat a funkciókat írják le, amelyre a felhasználó szemszögéből szükség van. A rendszer két különböző felületet tartalmaz: felhasználói- és adminisztrátori felület, ezek pedig különböző felhasználói követelményekkel rendelkezhetnek, így ezeket külön-külön részletezzük.

4.1.1. Felhasználói felület

A felhasználói felület funkcióinak nagy része bejelentkezés nélkül használható, kivételt képez a jegyvásárlás és a jegyek listázása. A felhasználói felület követelményei a következők:

- Legyen lehetőség utazást tervezni dátum, idő, kezdő- és célállomás alapján
- A talált járatok jelenjenek meg indulási idő szerinti növekvő sorrendben
- Legyen lehetősége a bejelentkezett felhasználónak jegyet vásárolni online a járatokra
- A rendszer jelenítse meg a buszok és megállók menetrendjeit
- A járatok útvonalait lehessen megtekinteni a térképen

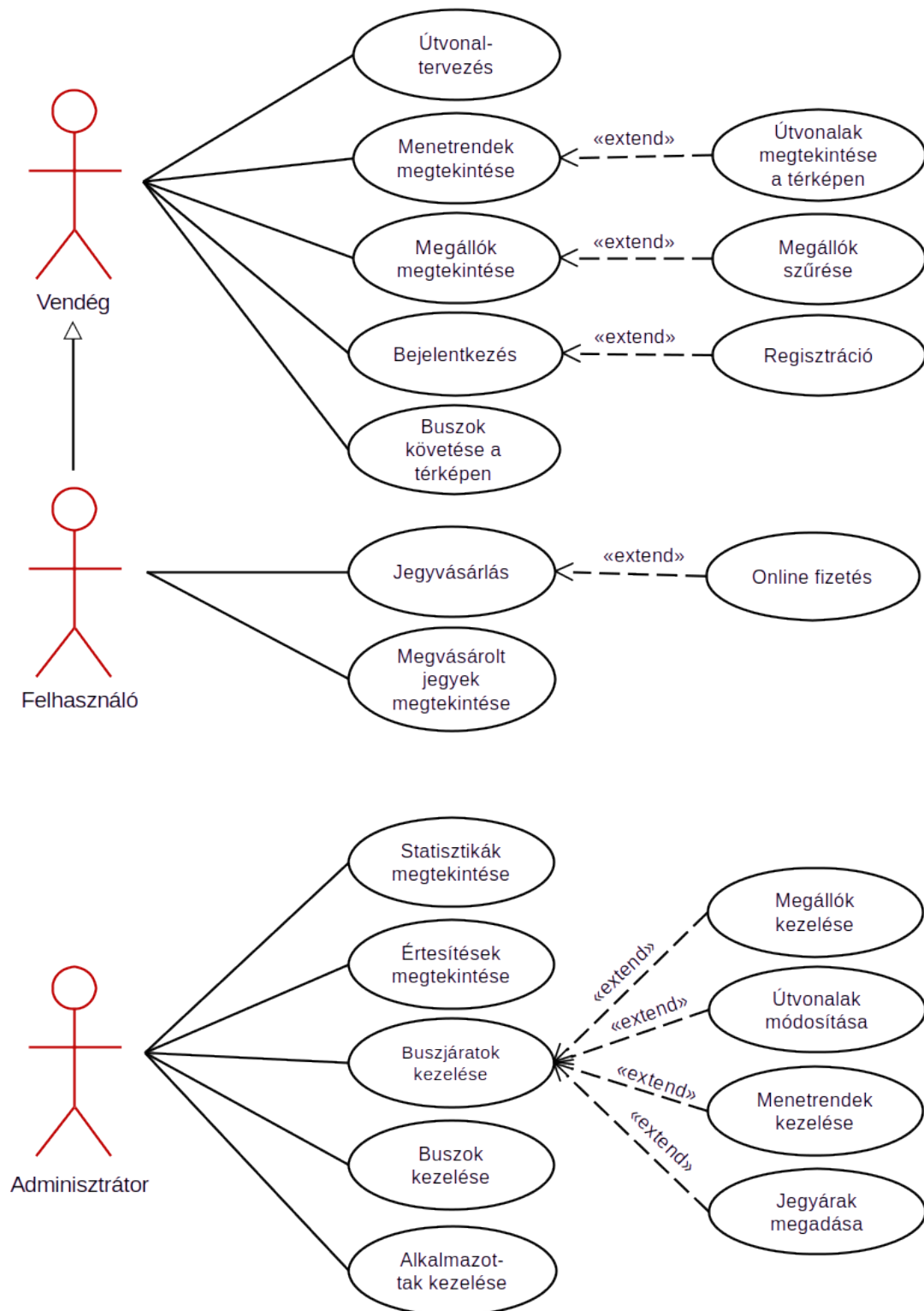
- Térképen megtekinthetők legyenek a megállók és szűrni is lehessen közöttük település vagy buszjárat alapján
- A rendszer biztosítsa a buszok valós idejű pozícióinak követését a térképen
- A QR-kódokkal ellátott megvásárolt jegyek megtekinthetők legyenek bejelentkezés után

4.1.2. Adminisztrátori felület

Az adminisztrátori felület kizárólag bejelentkezéssel elérhető, új fiókokat pedig csak a cégek főnökei tudnak létrehozni, mindenki a saját busztársaságához. Itt regisztrációs lehetőség nincs, így meg lehet akadályozni az adminfelülethez való illetéktelen hozzáférést. A társult vállalatok főnökei a partnerség kezdetekor megkapják az admin jogú fiókjaikat, amelyekkel hozzáférnek a felülethez. Az adminfelületre vonatkozó követelmények:

- A főoldalon emlékeztetők, értesítések, valamint statisztikák jelenjenek meg az eladott jegyek számáról, a megtett kilométerről és a bevételről, időintervallumok szerint csoportosítva
- Lehessen megtekinteni a megállók listáját, új megállókat hozzáadni térképen való megjelölés segítségével, valamint a meglévő megállókat szerkeszteni vagy törölni
- A rendszer listázza a buszjáratokat és biztosítson lehetőséget új járat létrehozására, meglévő járat szerkesztésére, valamint járatok törlésére
- A járatok indulási időpontjait lehessen megtekinteni, szerkeszteni és törölni is
- A buszjáratokhoz és az útvonalak különböző részeihez lehessen külön-külön jegyárat megadni
- A rendszer jelenítse meg a buszok listáját és biztosítson lehetőséget új busz hozzáadására, meglévő busz szerkesztésére, valamint buszok törlésére
- A buszokhoz hasonlóan lehessen kezelni az alkalmazotti fiókok listáját is

A rendszer funkcionalitását leíró használati eset diagram a 4.1 ábrán látható.



4.1. ábra. Use-case diagram a felhasználók lehetőségeiről

4.2. Rendszerkövetelmények

4.2.1. Funkcionális követelmények

A funkcionális követelmények a felületen megjelenő funkciók működését írják le. Ezek foglalják össze, hogy adott bemenetre milyen kimenetet kell szolgáltatnia a rendszernek. A Bus4U esetében ezeket is két csoportba sorolhatjuk: felhasználói és adminisztrátori követelmények.

A felhasználói weboldalon és a mobilalkalmazásban bejelentkezés nélkül hozzá lehet férni a főoldalhoz, a menetrendekhez, a megállókhoz és az élő térképhez. Az egyetlen oldal, ami nem jeleníthető meg azonosítás nélkül az a jegyek oldal, innen a rendszernek át kell irányítania a bejelentkezési felületre. Itt biztosítani kell a regisztráció lehetőségét is, amennyiben a felhasználó még nem rendelkezik fiókkal. A regisztráció során a rendszer le kell ellenőrizzen minden megadott adatot, még az email tulajdonjogát is, azáltal hogy egy ellenőrző kódot küld a megadott email címre, amelyet be kell írni a felületen megjelenő szövegmezőbe. Sikeres regisztráció vagy bejelentkezés után a rendszernek minden adatot biztonságosan el kell mentenie, a jelszavat hash-elte formában és át kell irányítania a felhasználót a főoldalra.

A főoldalon lévő szövegmezőkben megadott adatok alapján a rendszernek le kell kérnie a szerverről néhány lehetséges járatot, amelyek indulnak az adott településről, adott időben, a megadott célállomásra. A megjelenített találatokhoz tartoznia kell egy jegyvásárlási lehetőségnek is, amelyben meg kell adni a jegyek számát és véglegesíteni kell a vásárlást. A jegyvásárlás is bejelentkezést igényel, így az azonosítatlan felhasználót a rendszer vásárlás előtt átirányítja a bejelentkezéshez. A sikeres vásárlást követi a jegy kódjának kigenerálása és elmentése. Ezek után a felhasználók megtekinthetik a megvásárolt jegyeket a Jegyek oldalon. A menetrendek oldalon a felhasználóknak lehetőségük van a buszok és megállók közötti választásra lenyíló listákból, amelyet követően a rendszer kilistázza az adott járat megállóhoz kötött időpontjait egy táblázatban. Ugyanitt a rendszernek lehetőséget kell biztosítania a buszok útvonalának megjelenítésére térképen. A megállók oldalon a felhasználóknak lehetőségük van az összes megálló megtekintésére térképen, valamint lenyíló listákat használva, a megállók szűrésére település vagy buszjárat alapján is. A térképen megjelölt megállókra való kattintás után meg kell jelennie az adott megálló adatainak egy boborékban. Az élő térkép oldalon a felhasználóknak lehetőségük van a buszok valós idejű pozícióinak követésére térképen. A megjele-

nített buszokat a rendszernek valós időben kell frissítenie, hogy a felhasználók pontos információkat kapjanak a buszok helyzetéről. A térképek alapfunkciói közé tartoznak a nagyítás, a kicsinyítés és a mozgatás. A felhasználónak látnia kell a saját pozícióját is a térképeken, amennyiben engedélyezte a helymeghatározáshoz való hozzáférést.

Az adminisztrátori weboldalon szükség van azonosításra, anélkül a rendszer minden oldalról átirányít a bejelentkezéshez. Itt a regisztráció nem lehetséges, így külső, admin szerepkörrel nem rendelkező felhasználók nem tudnak hozzáférni az adminisztrátori felülethez. A főoldalon található statisztikák között szerepel az eladott jegyek száma, a megtett kilométerek száma és a bevétel összege, időintervallumok szerint csoportosítva, táblázatos formában. Mivel minden busztársaságnak el kell legyenek különítve az adatai, ezért a szerver mindig a bejelentkezett adminhoz tartozó cég adataiból számolja ki a statisztikákat. A főoldalon a rendszernek meg kell jelenítenie az esetleges értesítéseket is, előugró "Toast" üzenetek segítségével. A megállók oldalon az adminisztrátoroknak lehetőségük van a megállók listázására, az ezekre való egyenkénti kattintás után pedig a szerkesztésükre is. Emellett megállók törlésére és új megállók hozzáadására is lehetőség van egy interaktív térkép segítségével. A térképre kattintva a hely koordinátáit a rendszer beilleszti az űrlapba, ahol meg kell adni még a megálló nevét és a címet is. Utóbbi, amennyiben elérhető, le kell kérnie a rendszernek a térképszolgáltatótól, hogy ezzel is gyorsítsa a folyamatot. A járatok oldalon az adminisztrátoroknak lehetőségük van a buszjáratok kiválasztására egy lenyíló menüből, ezt követően pedig meg kell jelennie a járatszerkesztő űrlapnak. Ugyanitt lehetőség lesz járatok hozzáadására és törlésére is. A menetrendek oldalon az adminisztrátoroknak lehetőségük van a járatok indulási időpontjainak és jegyárainak megtekintésére, szerkesztésére és törlésére. Az órarendek bevitelét le kell lehessen bontani napokra, ezeket pedig dinamikusan megjeleníteni, hogy minél kevesebb helyet foglaljon az oldalon. A buszok és alkalmazottak oldalakon a rendszernek biztosítania kell egy egységes felületet, amelyben lehetőség nyílik az elemek listázására, új elemek hozzáadására, meglévő elemek szerkesztésére és ezek törlésére is. A buszoknál fontos megadni az útdó, biztosítás és műszaki vizsga időpontjait, mert a rendszer ezek alapján fog emlékeztetőket megjeleníteni a felületre való belépések alkalmával. Az alkalmazottaknál meg kell adni a nevet, a beosztást és a telefonszámot, valamint az emailt és a jelszót is. Utóbbiak az alkalmazott bejelentkezéséhez lesznek szükségesek, így a rendszernek ellenőriznie kell őket.

4.2.2. Nem funkcionális követelmények

Szervezéssel kapcsolatos követelmények

- Programozási nyelvek: Dart, HTML, CSS, JavaScript
- Keretrendszerek: Flutter, Vue.js
- IDE: Visual Studio Code
- Verziókövetés: Git és GitHub
- Webhost: Netlify
- Csoportos kommunikáció: Slack

Termékkel kapcsolatos követelmények

A felhasználói weboldal és az admin felület használatához egy modern, internetezésre alkalmas eszközre van szükség, a térképfunckciók teljes kihasználásához engedélyezni kell a helymeghatározást. A Vue.js keretrendszer minimum a JavaScript ES2016-os verzióját követeli meg, viszont a Vuetify komponensek miatt ajánlott a 2021 után kiadott böngészőverziók használata¹:

- Chromium (Chrome, Edge, Opera, Brave) ≥ 90
- Firefox ≥ 88
- Safari ≥ 15

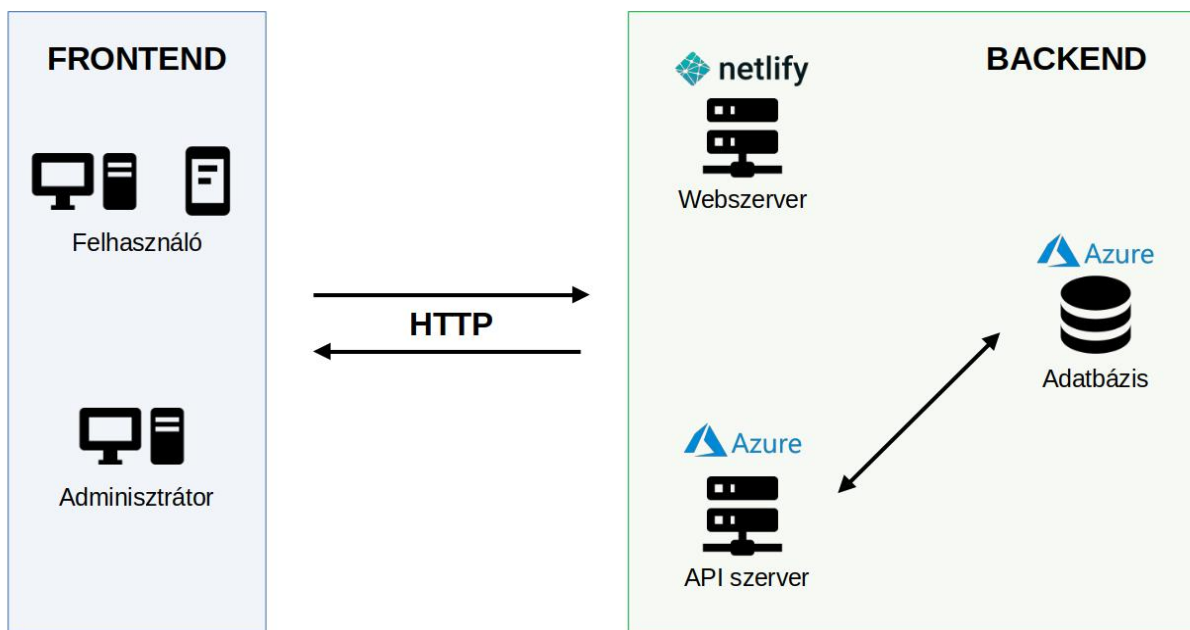
Az mobilapplikáció használatához szükség van egy okostelefonra vagy tabletre, amelyen minimum Android 5.0 vagy iOS 8.0 operációs rendszer fut. Fontos hogy legyen rajta internet elérés (3G,4G,5G vagy Wi-Fi), legalább 120MB szabad tárhely és 516-1024 MB szabad memória. Itt is ajánlott a helymeghatározás (GPS) engedélyezése, hogy a térképfunckciók helyesen működjenek.

¹ Vuetify böngésző támogatás: <https://vuetifyjs.com/en/getting-started/browser-support/>

5. fejezet

A rendszer architektúrája

Az általunk tervezett rendszer architektúra diagramja a 5.1 ábrán látható. A rendszer frontend és backend része jól elkülöníthető egymástól, az egyes komponensek közötti kommunikáció titkosított HTTP kérések segítségével valósul meg. Először részletezzük a frontend és a backend felépítését, majd a közöttük megvalósuló kommunikációt.



5.1. ábra. A rendszer architektúrája

Egyes oldalakon térképek is megjelennek, amelyeket a web esetében a MapBox, míg a mobil esetében a Google Maps szolgáltat. Mindkét esetben az adott platformhoz és technológiához közelebb álló térképszolgáltatót választottuk, hogy a lehető legjobb felhasználói élményt tudjuk biztosítani.

5.2. Backend

A backend a rendszer azon része, amely a kliens oldali felületek mögött fut, és biztosítja az adatok kezelését, tárolását, valamint az üzleti logika végrehajtását. A jelenlegi rendszerben a backend egy REST architektúrát követő API szerverként működik, amely Ruby on Rails technológiával készült. Ez az API szerver szolgálja ki mind a két weboldalt (adminisztrátori és végfelhasználói), valamint a mobilalkalmazást is.

A kliens oldali alkalmazások közvetlenül kommunikálnak a backend API végpontjaival, mindenféle köztes logika nélkül, így a rendszer architektúrája egyértelműen a kliens-szerver modellre épül.

A token alapú hitelesítést a rendszer a Devise felhasználókezelő könyvtár kiegészítéseként a devise-jwt gem segítségével valósítja meg. Ez lehetővé teszi a biztonságos, állapotmentes (stateless) autentikációt az API-n keresztül, anélkül hogy a szervernek külön munkameneteket (session-öket) kellene tárolnia.

A backend további feladatai közé tartozik az útvonalak, jegyek, bérletek, felhasználók, járművek és egyéb erőforrások adatainak kezelése, validálása és naprakészen tartása. A backend az adatokat egy PostgreSQL alapú relációs adatbázisban tárolja. Az adatok integritását és konzisztenciáját az ActiveRecord ORM réteg biztosítja, amely a Rails keretrendszer része.

A rendszer biztonsága érdekében az API végpontokhoz való hozzáférést szerepkör-alapú jogosultságkezeléssel szabályozzuk. Az adminisztrátorok számára elérhető végpontok például statisztikákat, jelentéseket és különféle menedzsment funkciókat biztosítanak, míg a végfelhasználók csak a saját jegyeikhez, bérleteikhez és a releváns útvonal-információkhoz férhetnek hozzá.

5.3. Kommunikáció

A felek közötti kommunikáció REST API hívásokkal valósul meg, ezt szemlélteti az 5.4 ábrán megjelenített szekvencia diagram, amely a felhasználó által megvásárolt jegyek listázásának folyamatát írja le. Amikor a felhasználó a frontend felületen keresztül a My Tickets oldalra navigál, egy GET kérés indul el a szerver felé, amely tartalmazza a felhasználó adatait. A backend először ellenőrzi a kérést, majd a megfelelő jegyeket lekérdezi az adatbázisból és visszaküldi a frontendnek, ahol azok idő szerinti rendezés után megjelennek a felhasználó számára. A még érvényes jegyekhez generál a rendszer egy QR kódot is, hogy könnyű legyen majd innen beolvasni. Amennyiben valahol hiba történik, a rendszer informálja erről a felhasználót. A következő kódrészlet intézi a kérést a frontend oldalról:

```
isLoading.value = true
axios.get('https://api.bus4u.online/v1/tickets_of_user', { headers: { Authorization: user.authorization }, params:
  { uid: user.uid }})
  .then(rsp => {
    isLoading.value = false
    tickets.value = rsp.data.sort((a, b) => {
      if(!a.is_valid && b.is_valid) return 1
      if(!b.is_valid && a.is_valid) return -1
      return b.date_of_purchase.localeCompare(a.date_of_purchase)
    })
    for(let i=0; i<tickets.value.length; i++) {
      if(!tickets.value[i].is_valid) continue
      QRCode.toDataURL(tickets.value[i].ticket_uid, { width: 250 }).then(rsp => tickets.value[i].qr = rsp)
    }
  }).catch(() => {
    tickets.value = []
    isLoading.value = false
  })
```

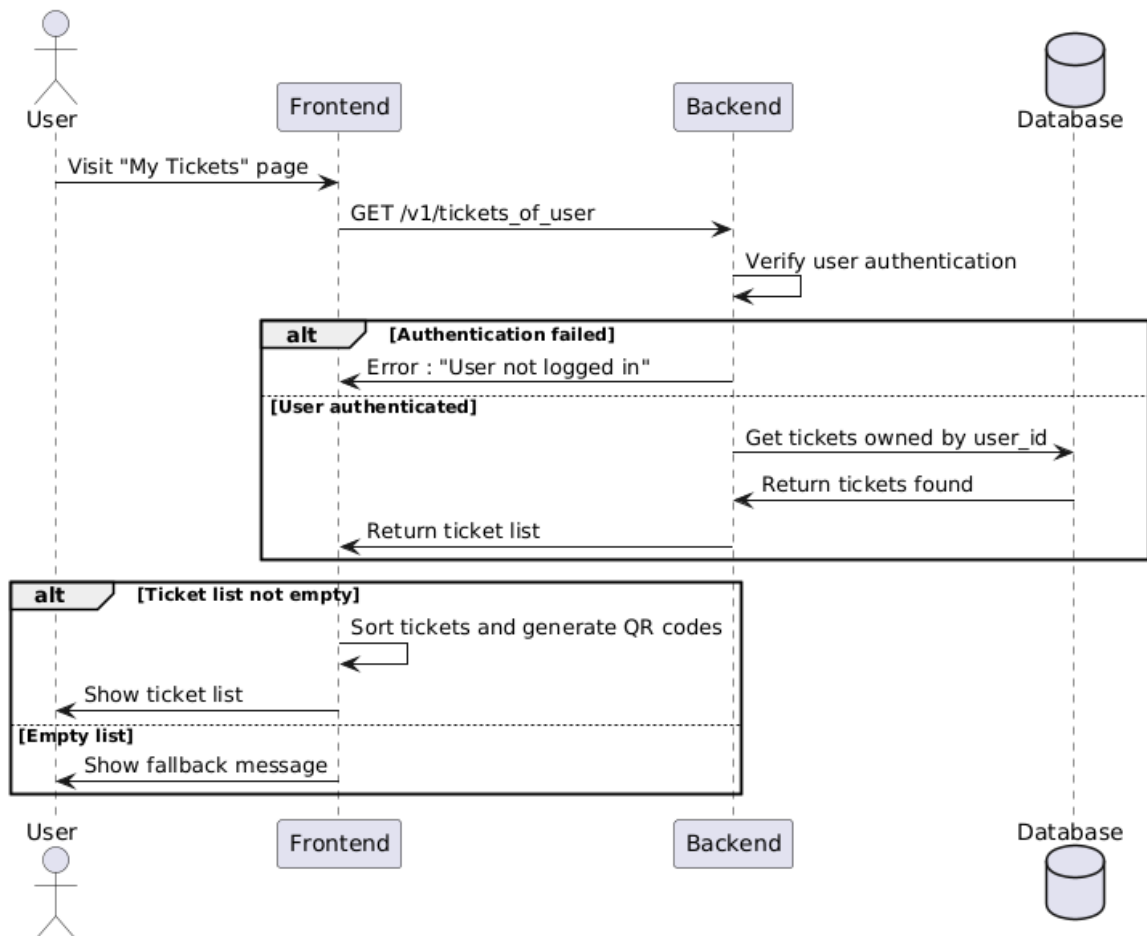
Egy ilyen kérésre, a backend által adott válasz az 5.3 ábrán látható.

```

JSON
▼ 0: Object { ticket_uid: "TIC_ECB5K", date_of_purchase: "2024-10-11T17:25:26.981+03:00", expiration_date: "2024-11-11T16:25:26.981+02:00", ... }
    ticket_uid: "TIC_ECB5K"
    date_of_purchase: "2024-10-11T17:25:26.981+03:00"
    expiration_date: "2024-11-11T16:25:26.981+02:00"
    from_station_uid: "STT_EV9LM"
    to_station_uid: "STT_PXVPC"
    from_station_name: "Izvor"
    to_station_name: "Grand"
    is_valid: true
    route_uid: "ROU_SOXDN"
    route_name: "26"
    ticket_price: "200.0"
► 1: Object { ticket_uid: "TIC_SK8EA", date_of_purchase: "2024-10-11T17:25:27.069+03:00", expiration_date: "2024-11-11T16:25:27.069+02:00", ... }
► 2: Object { ticket_uid: "TIC_AG9KQ", date_of_purchase: "2024-10-11T17:29:37.005+03:00", expiration_date: "2024-11-11T16:29:37.005+02:00", ... }

```

5.3. ábra. A jegyek listázásának válasza



5.4. ábra. A jegyek listázásának folyamata

Ugyanígy leírható a regisztráció folyamata is, ahol a felhasználó megadja a nevét, az email címét és az új jelszót, majd a rendszer az adatok ellenőrzése után elküldi egyelőre csak az email címet a backend felé. A backend az email szerver segítségével küld egy 4 számjegyű kódot a felhasználó által megadott email címre. Ezután a felhasználó be kell írja ezt az ellenőrző kódot a felületen ennek megfelelő helyre. Ezt a kódot a backend ellenőrzi és amennyiben helyes, a frontend elküldi a felhasználó további adatait is, amelyet a backend elment az adatbázisban. Ha minden lépés sikeresen végrehajtódik, a frontend elmenti a bejelentkezési adatokat kliens oldalon is és átirányítja a felhasználót a főoldalra. Ellenkező esetben hibaüzenet jelenik meg a lépéssorozat azon pontján, ahol a hiba keletkezett. A regisztráció utolsó lépését (felhasználói adatok elküldése) megvalósító kódrészlet a következő:

```
async function sendForm() {
  axios.post('https://api.bus4u.online/auth', form).then((rsp) => {
    if(rsp.data.data.uid) {
      router.replace({ name: 'home' })
      user.signIn(rsp.data.data, rsp.headers)
    } else {
      isLoading.value = false;
    }
  }).catch((e) => {
    if(e.response) error.value = e.response.data.error;
    isLoading.value = false;
  });
}
```

JSON

```
status: "success"
data: Object { provider: "email", uid: "[REDACTED]@student.ms.sapientia.ro", allow_password_change: false, ... }
```

provider: "email"

uid: "[REDACTED]@student.ms.sapientia.ro"

allow_password_change: false

firstname: "[REDACTED]"

lastname: "[REDACTED]"

email: "[REDACTED]@student.ms.sapientia.ro"

phone_number: null

language: null

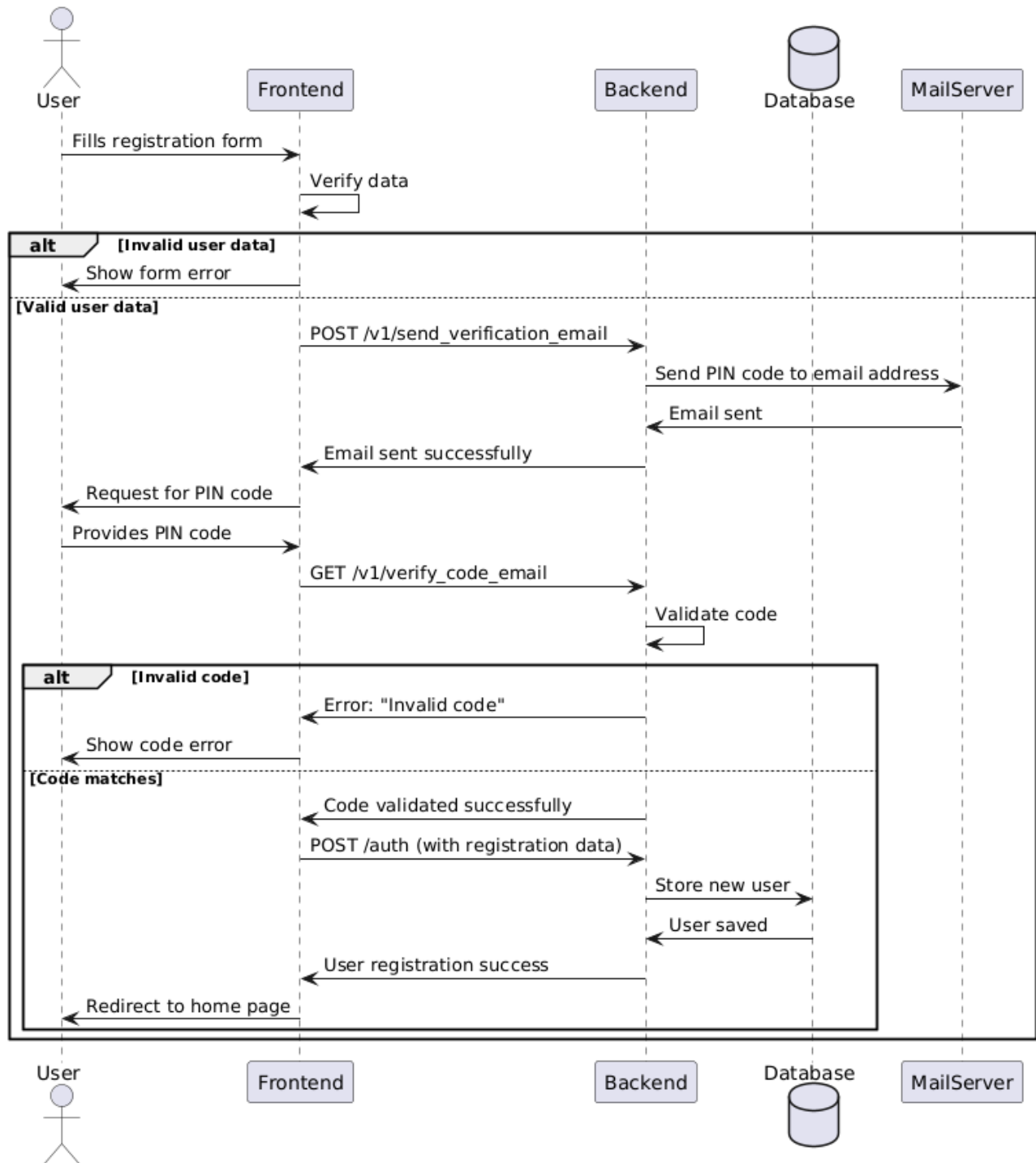
stripe_id: "[REDACTED]"

created_at: "2025-02-19T16:48:25.272+02:00"

updated_at: "2025-02-19T16:48:25.363+02:00"

5.5. ábra. A regisztrációra adott válasz

A regisztrációs adatok elküldésére adott válasz az 5.5 ábrán látható, míg a teljes regisztrációs folyamatot szemléltető szekvencia diagram az 5.6 ábrán látható.



5.6. ábra. A regisztráció teljes folyamata

5.4. Felhasznált technológiák

5.4.1. Frontend

A felhasználói és az admin weboldalak Vue.js keretrendszerben készültek, Vuetify komponenseket felhasználva. Azért esett a választás erre a kombinációra, mert a Vue által gyors és hatékony weboldalakat lehet létrehozni, a Vuetify pedig tovább gyorsítja a fejlesztést előre elkészített, jól kinéző komponensek szolgáltatásával. A mobilra szánt alkalmazás elkészítéséhez Flutter technológiát használtunk, mert ennek segítségével egyetlen kódbázisból ki lehet generálni Android és iOS készülékekre is az alkalmazást, amely hatékonyan működik minden eszközön a keretrendszer optimalizációi miatt.

Vue.js és Vuetify

A Vue.js egy progresszív JavaScript keretrendszer, amelyet azért hoztak létre, hogy a webes felhasználói felületek fejlesztése egyszerűbb és gyorsabb legyen. A Vue.js abban különbözik a többi JavaScript keretrendszertől, hogy egy könnyű, gyors és hatékony technológiát kínál, amely egy Virtuális DOM modell segítségével gyorsítja a futást és amelynek a forráskódja akár 24 Kb-ra is zsugorítható, így viszonylag gyors a betöltése is [8].

A Vue.js két alapköve a deklaratív UI felépítés és a reaktivitás. Az első azt jelenti hogy az alap HTML kódot a Vue kiegészíti egy saját sablon-szintaxissal, amely lehetővé teszi a változók beágyazását a HTML kódba. A reaktivitás azt jelenti, hogy amikor a HTML sablonban felhasznált JavaScript változók értéke megváltozik, akkor a Vue érzékeli ezt és automatikusan megváltoztatja a megjelenített HTML kódot, így mindig a legfrissebb adatokat látja a felhasználó. Ugyanez végbemegy fordított irányban is, mert a felhasználói interakciók eredményeképpen a Vue automatikusan frissíti a háttérben lévő változókat is, így lesz a kommunikáció a DOM és a Vue között kétirányú [8, 9].

A Vue keretrendszer egyik legnagyobb előnye, hogy könnyen tanulható és alkalmazható, így gyorsan lehet vele fejleszteni. Ennek egyik oka, hogy komponens alapú, amely lehetővé teszi a weboldalak felépítését újrafelhasználható, különálló komponensekből, amelyeket később össze lehet fűzni egy nagyobb alkalmazássá [9]. Emellett a Vue nagyon jó online dokumentációval rendelkezik, amelyben minden funkció részletesen le van írva, de egyéb problémákkal és kérdésekkel a dinamikusan növekvő fejlesztői közösséghez is lehet fordulni [8].

A felhasználó szemszögéből a leglátványosabb tulajdonság, hogy nagyon gyors a navigáció, mivel a Vue Single Page Application üzemmódja által csak egyszer töltődik be a webalkalmazás, utána az oldalak közötti navigáció során csak a tartalom frissül, nem pedig az egész oldal, így kevesebb adat közlekedik a hálózaton és a böngészőnek is könnyebb dolga van a megjelenítésnél [9].

Fontos megemlíteni a Vue ökoszisztémájába tartozó egyéb eszközöket is, amelyek segítik a fejlesztést. Ilyen például a Vue Router, amely a weboldalak közötti navigációt teszi lehetővé, a Pinia, amely a globális állapotkezelést segíti, a Vite, amely a projekt generálását és a fejlesztést segíti [9]. Léteznek UI könyvtárak is, például a Vuetify, amely lehetővé teszi a gyors és egyszerű felhasználói felületek készítését, azáltal hogy rengeteg előre elkészített, Material Design alapú Vue komponenset tartalmaz, amelyeket egyszerűen be lehet integrálni a weboldalba, így sok időt lehet vele spórolni. Az itt felsorolt technológiák segítségével gyorsan és hatékonyan sikerült felépíteni a weboldalak felhasználói felületeit, mi több, ezek jól néznek ki és jól is működnek minden tesztelt eszközön.

Flutter

A Flutter egy nyílt forráskódú, Google által kifejlesztett, cross-platform, UI készítő keretrendszer, amely lényege, hogy egyetlen kódbázisból ki lehet generálni az alkalmazásokat a legnépszerűbb platformokra. Ezek a platformok a következők: Android, iOS, Windows, macOS, Linux, valamint a web.

A Flutter a Dart programozási nyelvet használja, amely eredetileg a JavaScript lecserélésére volt létrehozva a webfejlesztésben, viszont idővel a Flutter technológia alapjává vált és így tett szert népszerűsége. A Flutter egyik nagy előnye, hogy egy widget rendszert használ, amely lehetővé teszi a felhasználói felületek egyszerű és gyors elkészítését. Ezek a widgetek alapértelmezetten a Material Designra épülnek, amely összhangban van a Vuetify webes komponenseivel is. A Flutter egy másik nagy előnye a hatékonyság, mivel natív kódra fordítja le a Dart kódot, így a lehető legjobb teljesítményt nyújtja, bármilyen platformról is legyen szó [10].

Nagyban megkönnyíti a mobilra történő fejlesztést a Hot Reload, mivel a kódban történt változásokra azonnal reagál a felhasználói felület, így nem kell mindig újraindítani az alkalmazást [10]. A Flutterben való alkotás hatékonyságát tovább növeli, hogy részletes online dokumentációval rendelkezik, és hogy a Pub Package Manager-t (Csomagkezelőt) használja, amely rengeteg előre elkészített csomagot tartalmaz, az egyszerű UI komponensektől kezdve, a komplex háttérfolyamatokig.

5.4.2. Backend

A szerveroldali logika Ruby on Rails keretrendszer segítségével készült, amely lehetővé teszi a gyors fejlesztést és az átlátható, strukturált kódbázis kialakítását. A Rails erőssége, hogy számos beépített funkcióval rendelkezik, amelyek megkönnyítik az API-k létrehozását, az adatkezelést és a biztonságos működést. Az adatbázis kezelésére PostgreSQL-t választottunk, mivel megbízható, nyílt forráskódú relációs adatbázis-kezelő, amely jól skálázható és támogatja az összetett lekérdezéseket. A backend kiszolgálása az Azure felhőszolgáltatásban történik, amely rugalmasságot, automatikus skálázást és folyamatos rendelkezésre állást biztosít az alkalmazás számára.

Ruby on Rails

A Ruby on Rails egy népszerű és széles körben használt webes keretrendszer, amely a Ruby programozási nyelvre épül. A Rails két alapvető elv alapján működik: a „*Convention over Configuration*” és a „*Don't Repeat Yourself*”. Ezek az elvek azt eredményezik, hogy a fejlesztőknek kevesebb kóddal kell dolgozniuk, mivel a Rails konvenciói automatikusan meghatározzák a konfigurációkat, így csökkentve a redundáns kódot és növelve a fejlesztési folyamat hatékonyságát [11, 12]. Ez különösen hasznos nagyobb projektek esetén, mivel elősegíti a gyorsabb fejlesztést és az egyszerűbb karbantartást, amely a gyorsan változó üzleti igények mellett nélkülözhetetlen.

A Rails *MVC (Model-View-Controller)* architektúrára épül, amely logikailag elkülöníti az alkalmazás különböző részeit: az adatkezelést (Model), a felhasználói felületet (View), és az üzleti logikát (Controller). Ez a struktúra átláthatóbbá teszi a kódot, mivel a különböző funkciók külön-külön karbantarthatók, anélkül hogy zavarnák egymást, valamint megkönnyíti az új funkciók hozzáadását is. A Rails alkalmazásokhoz számos „gem”, vagyis könyvtár érhető el, amelyek előre elkészített megoldásokat kínálnak, így még a komplexebb funkciók integrálása is könnyebb és biztonságosabb lehet [11].

A Rails további előnye, hogy natív módon támogatja *RESTful API*-k létrehozását. A RESTful megközelítés egyértelmű, szabványosított formát biztosít az API-k számára, amely megkönnyíti az alkalmazások közötti integrációt, például egy mobilalkalmazás és a backend közötti kommunikáció során. Ennek a felépítésnek köszönhetően a Rails alkalmazások egyszerűen összekapcsolhatók más

rendszerekkel és frontend megoldásokkal [12].

PostgreSQL

A PostgreSQL egy nyílt forráskódú, objektum-relációs adatbázis-kezelő rendszer, amely széles körben ismert teljesítményéről, rugalmasságáról, és különösen alkalmas komplex adatszerkezetek és nagy adattömegek kezelésére. Támogatja a fejlett adattípusokat, bonyolult lekérdezéseket, és párhuzamos műveleteket, amelyek különösen hasznosak a Bus4U-hoz hasonló, valós idejű adatokat feldolgozó rendszerekben [13, 14].

A PostgreSQL kiválóan kezeli a párhuzamos lekérdezéseket és optimalizált módon képes kezelni nagy mennyiségű adatot és komplex lekérdezési struktúrákat. Ez biztosítja az alkalmazás gyors, megbízható működését, és lehetőséget nyújt az adatbázis hatékony méretezésére és finomhangolására az adatmennyiség növekedésével [14].

Nagyobb adatmennyiség vagy felhasználói terhelés esetén a PostgreSQL támogatja a függőleges és vízszintes skálázást. Ezen túlmenően replikációval biztosítható a magas rendelkezésre állás, amely kritikus szempont a nagy terhelés alatt működő rendszerekben. Így a PostgreSQL hosszú távú, fenntartható teljesítményt és magas rendelkezésre állást biztosít, amely elengedhetetlen a Bus4U megbízható működéséhez [13].

Azure

Az Azure egy olyan nagyvállalati szintű, biztonságos és skálázható felhőplatform, amely számos lehetőséget kínál modern alkalmazások fejlesztéséhez és működtetéséhez. Az Azure infrastruktúrája révén biztosítható a folyamatos és megbízható működés, míg a platform rugalmassága lehetővé teszi az erőforrások hatékony kezelését és a skálázhatóságot [15, 16]. Az Azure használatával elkerülhetőek a hagyományos szerver infrastruktúrák fizikai korlátai, és olyan erőforrások válnak elérhetővé, amelyek az alkalmazás igényei szerint rugalmasan méretezhetők.

Az Azure automatikusan alkalmazkodik a változó terheléshez, amely lehetővé teszi, hogy a rendszer gördülékenyen kezelje a megnövekedett felhasználói forgalmat. Ez az adaptív megközelítés egy modern, folyamatosan növekedő alkalmazás számára elengedhetetlen, különösen tömegközlekedési rendszerek esetében, ahol a felhasználói igények időben változhatnak [15].

Az Azure erőteljes adatvédelmi szabályokat és fejlett biztonsági protokollokat alkalmaz. Ezek közé tartozik a többfaktoros hitelesítés, a hálózati tűzfalak, valamint az adattitkosítás, amelyek biztosítják, hogy a Bus4U felhasználóinak adatai biztonságban legyenek. A platform megfelel a szigorú adatvédelmi szabványoknak, ami kulcsfontosságú egy olyan alkalmazás számára, amely szenzitív felhasználói adatokat kezel [16].

5.5. Fejlesztői eszközök

A fejlesztés során számos eszközt használtunk, amelyek segítették a hatékonyságot. A legfontosabb ezek közül a kódszerkesztő, amelynek a Visual Studio Code-ot használtuk, mert ez egy könnyű, gyors és hatékony eszköz, amely jól testreszabható a rengeteg letölthető kiegészítő által, amelyeknek köszönhetően támogat szinte bármilyen programozási nyelvet és keretrendszert².

Mivel a projekten ketten dolgoztunk, szükség volt egy eszközre amelyben vezetni tudjuk a feladatokat átlátható módon. Erre a Jira-t használtuk, amely egy online projektmenedzsment eszköz, amelynek segítségével könnyen lehet követni a feladatokat, a fejlesztés állapotát és a határidőket is egy kanban board-on.³ Itt felbontottuk az egész projektet kisebb feladatokra, úgynevezett taszkokra, amelyekhez kódneveket rendeltünk hozzá. Ezeknek a formája *BUS-XX* volt, ahol az *XX* egy sorszám, amely a feladatok sorrendjét jelöli. A taszkok nevei tartalmazták a feladat rövid leírását, de hozzá lehetett adni bővebb leírást és hozzászólásokat is. A 5.7 ábrán látható az általunk használt Jira tábla.

Verziókövetésre a Git-et használtuk, GitHub-al kiegészítve, így biztonságosan tudtuk tárolni a kódot és a változtatásokat is könnyen nyomon tudtuk követni. A GitHub abban is segített, hogy összehangoljuk a frontend és a backend fejlesztését, valamint hogy több, különböző eszközről is tudjunk dolgozni. A munka során branch-ekre (ágakra) osztottuk a projektet, úgy hogy a fő ág mellett volt egy develop nevű ág és ebből váltak ki az egyes feladatok mellékágai, amelyek mind egy-egy Jira taszknak feleltek meg. A mellékágak nevei tartalmazták a taszk kódját, így össze lehetett hangolni a verziókövetést a kanban táblával. A branchek használata megkönnyítette és strukturáltabbá tette a közös munkát.

²Visual Studio Code: <https://code.visualstudio.com/>

³A projekt Jira oldala: <https://bus4u.atlassian.net/jira>

Projects / Bus4U

Issues

Share Export Go to all issues LIST VIEW DETAIL VIEW ...

Search issues Project = Bus4U Type Status Assignee = Portik Szabolcs (+1) More + Go back to filter Save filter BASIC JQL

Type	Key	Summary	Assignee	Reporter	Priority	Status	Resolution	Created	
☒	BUS-61	Preparations for automatic filling of waybills	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	TO DO	Unresolved	Oct 20, 2024	Oct
☒	BUS-60	Redesign Flutter app	PS Portik Szabolcs	PS Portik Szabolcs	Medium	IN PROGRESS	Unresolved	Oct 11, 2024	Oct
☒	BUS-59	Preparations for redeployment on Azure	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Oct 3, 2024	Oct
☒	BUS-58	Fix application ticket buying error	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Jan 11, 2024	Jan
☒	BUS-56	Final frontend bugfixes	PS Portik Szabolcs	PS Portik Szabolcs	Medium	DONE	Done	Jan 3, 2024	Jan
☒	BUS-55	Update methods for admin requests	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Jan 3, 2024	Jan
☒	BUS-54	Add token validation to pos_app	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Dec 20, 2023	Jan
☒	BUS-51	Request for the number of remaining bus stops	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	TO DO	Unresolved	Dec 19, 2023	Jan
☒	BUS-50	Update methods for elements	PS Portik Szabolcs	PS Portik Szabolcs	Medium	DONE	Done	Dec 19, 2023	Jan
☒	BUS-49	Create buses page	PS Portik Szabolcs	PS Portik Szabolcs	Medium	DONE	Done	Dec 18, 2023	Dec
☒	BUS-48	Write a base for the documentation	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Dec 17, 2023	Dec
☒	BUS-47	Fix POS terminal app bugs	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Dec 14, 2023	Dec
☒	BUS-46	Implement actionmailer for the tickets	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Dec 13, 2023	Dec
☒	BUS-45	Create employees page	PS Portik Szabolcs	PS Portik Szabolcs	Medium	DONE	Done	Dec 13, 2023	Dec
☒	BUS-44	Final touches on the aoi	ZA Zsigmond Attila	ZA Zsigmond Attila	Medium	DONE	Done	Dec 11, 2023	Jan

1-49 of 49 1

5.7. ábra. Feladatok a Jira-ban

5.6. Kitelepítés

5.6.1. Azure infrastruktúra áttekintése

A Bus4U rendszer a Microsoft Azure felhőalapú platformján került telepítésre. Az Azure szolgáltatások skálázhatóságot és megbízhatóságot biztosítanak a rendszer számára. A telepítéshez az alábbi főbb szolgáltatásokat használtuk:

- Azure Virtual Machines (VM): A szerver hosztolása.
- Azure Database for PostgreSQL: Az adatbázis-szolgáltatás biztosítása.

5.6.2. Domain konfiguráció

A rendszer végpontjai az api.bus4u.online domainen érhetőek el, amelyet a Cloudflare platformján konfiguráltam. A domain hozzárendelése során a virtuális gép (VM) IP címét használtam. A következő beállításokat végeztem el:

- DNS beállítások: A Cloudflare-en elvégeztem a domain névhez tartozó A és CNAME rekordok konfigurálását.
- SSL tanúsítványok: A HTTPS alapú kapcsolat biztonságának érdekében a Cloudflare szolgáltatásait alkalmazzuk, amelyek védelmet nyújtanak a felhasználók számára.

5.6.3. Adatbázis telepítése és konfigurációja

A rendszer adatbázisa egy Azure Database for PostgreSQL szolgáltatáson keresztül került telepítésre, amely lehetővé teszi az adatbázis automatikus méretezését és a folyamatos biztonsági mentések készítését. A következő beállításokat alkalmazzuk:

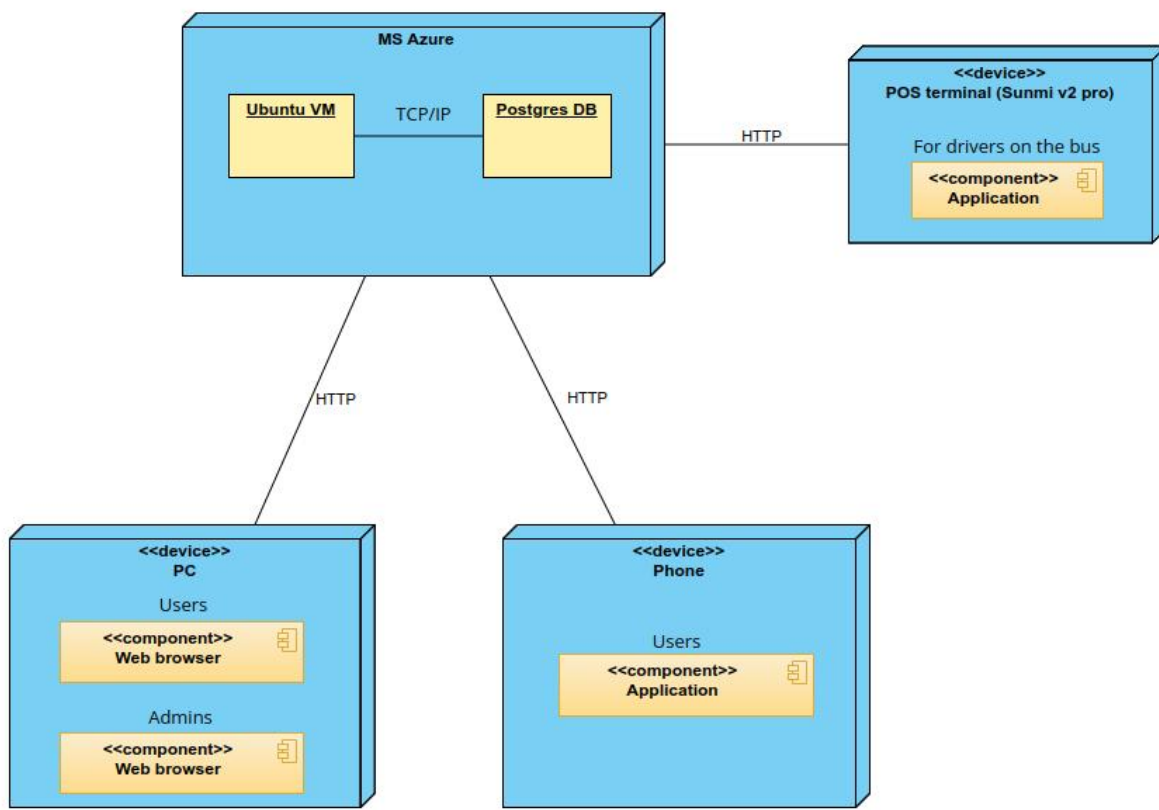
- Titkosított kapcsolatok: A PostgreSQL adatbázis SSL tanúsítványokat használ a biztonságos kommunikációhoz.
- Backup és recovery: Automatikus biztonsági mentések kerülnek tárolásra az adatvesztés elkerülése érdekében.

5.6.4. Szerver telepítése

A szerver az Azure Virtual Machines szolgáltatásban került telepítésre, ahol a Ruby on Rails alapú API-t futtatjuk. A telepítés során a következő lépéseket hajtottuk végre:

- Környezeti változók beállítása: A szükséges konfigurációs változók (pl. adatbázis kapcsolatok) a Ruby on Rails beépített credentials funkciójában lettek biztonságosan tárolva és kezelve.
- Verziókezelés: Az asdf verziókezelő használatával a Ruby és egyéb nyelvek verzióit könnyen kezelhetjük, ami megkönnyíti a fejlesztési környezet fenntartását.
- CI/CD pipeline: Az alkalmazás automatikus telepítése és frissítése a GitHub Actions integrációjával valósult meg.

A részletes telepítési folyamatot a 5.8 ábra szemlélteti.



5.8. ábra. Telepítési diagram

A weboldalak ki lettek telepítve Netlify-ra, hogy könnyebben tesztelhetőek és bemutatathatóak legyenek. A Netlify egy olyan szolgáltatás, amely lehetővé teszi a statikus weboldalak gyors és egyszerű kiszolgálását a GitHub repository-ból, így a változtatások a *git push* parancs után azonnal megjelennek a weben. A felhasználói weboldal a <https://bus4u.online>, az adminisztrátori weboldal pedig a <https://admin.bus4u.online> címen érhető el.

6. fejezet

Adatkezelés

6.1. API végpontok részletes bemutatása

A backend végpontjait a Postman alkalmazás segítségével követtük nyomon, amely lehetővé teszi HTTP-alapú API-k vizuális kezelését, tesztelését és dokumentálását. A fejlesztés során a Postmant nemcsak hibakeresésre, hanem a végpontok működésének gyors ellenőrzésére és az API tervezés vizualizálására is használtuk. Lehetőséget ad az egyes kérések testreszabására (pl. fejlécek, paraméterek, autentikáció), valamint az API válaszainak könnyű értelmezésére.

- **GET /v1/get_stations_by_city** – Ez az API végpont lehetővé teszi, hogy egy adott városhoz tartozó állomásokat kérjünk le. Ezen végpont segítségével a felhasználók gyorsan megtalálhatják azokat az állomásokat, amelyek egy meghatározott városhoz tartoznak. A kérés paraméterei tartalmazzák a város nevét, és a válasz egy listát ad vissza az állomásokról, amint azt a 6.1 ábra is mutatja.

Query Params			
☒ Key	Value	Description	... Bulk Edit
☒ city_uid	CTY_2ZGHU		
Key	Value	Description	

Body	Cookies	Headers (14)	Test Results	🔍	200 OK	• 4.06 s • 1.1 KB • 🌐	📄 Save Response	...
------	---------	--------------	--------------	---	--------	-----------------------	-----------------	-----

{} JSON
 ▶ Preview
 🌐 Visualize
 |

▼

▼ stations [3]

	station_uid	name	address	longitude	latitude
0	STT_9CVLG	Gyergyó Állomás	Str. Garil nr. 16	25.5740503955354	46.71952175752869
1	STT_XANQM	Gyergyó Maros hotel	Bul. Fratiei nr. 65	25.59475044928773	46.72267896566973
2	STT_JL205	Gyergyó Központ	Pta. Libertatii nr. 26	25.59998843540028	46.72080260302078

6.1. ábra. A GET /v1/get_stations_by_city végpont kérésének paraméterei és válasza

- **GET /v1/get_available_tickets** – A 6.2 ábrán is látható módon ez végpont lehetővé teszi az elérhető jegyek lekérdezését egy adott kiindulási és célállomás között. A kérés paraméterei a kiindulási és célállomás, valamint egy opcionális időpont. A megállók is opcionálisak, a csak a város megadása esetén minden megállóhoz láthatjuk a jegyeket. A válasz tartalmazza az elérhető jegyek listáját, és ezek árai.

Query Params			
☐ Key	Value	Description	... Bulk Edit
☒ start_city_uid	CTY_2ZGHU		
☐ start_station_uid	STT_D2Y7W		
☒ destination_city_uid	CTY_CCTK2		
☐ destination_station_uid	STT_O172E		
☒ date	2023-12-21		
☒ time	16:00		
Key	Value	Description	

Body	Cookies	Headers (14)	Test Results	🔍	200 OK	• 6.36 s • 1.84 KB • 🌐	📄 Save Response	...
------	---------	--------------	--------------	---	--------	------------------------	-----------------	-----

{} JSON
 ▶ Preview
 🌐 Visualize
 |

▼

	from_station	from_station_uid	to_station	to_station_uid	company_name	company_uid	route_name	route_uid	ticket_price	departure_time
0	Gyergyó Állomás	STT_9CVLG	Dedeman	STT_W848K	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_J0BF3	38.0	1743098700
1	Gyergyó Állomás	STT_9CVLG	Plata Trandafirilor	STT_RNUUP	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_J0BF3	39.0	1743098700
2	Gyergyó Központ	STT_JL205	Dedeman	STT_W848K	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_J0BF3	40.0	1743098400
3	Gyergyó Központ	STT_JL205	Plata Trandafirilor	STT_RNUUP	VANDOR TRANS TOURS SRL	CPY_MNPCM	Gyergyó-Marosvásárhely	ROU_J0BF3	41.0	1743098400

6.2. ábra. A GET /v1/get_available_tickets végpont kérésének paraméterei és válasza

- **GET /v1/get_bus_locations_by_route** – Ezen API végpont segítségével lekérhetjük a buszok aktuális helyzetét egy adott járatra vonatkozóan. A kérés tartalmazza a járat azonosítóját, és a válasz egy koordinátákkal mutatja a buszok aktuális pozícióit, amint azt a 6.3 ábra is mutatja.

Query Params

☒ Key	Value	Description	...	Bulk Edit
☒ route_uid	ROU_I0BF3			
Key	Value	Description		

Body Cookies Headers (14) Test Results | 200 OK • 591 ms • 1.08 KB | Save Response

{ } JSON Preview Visualize

	bus_uid	company_uid	license_plate	brand	manufacturing_year	capacity	road_tax	insurance	technical_exam
0	BUS_GDP1W	CPY_MNPCM	HR12VTT	Otokar Sultan	2011	24	2024-06-29T00:00:00.000+03:00	2024-01-25T00:00:00.000+02:00	2024-06-11T00:00:00.000+03:00

6.3. ábra. A GET /v1/get_bus_locations_by_route végpont kérésének paraméterei és válasza

- **GET /v1/tickets_of_user** – Az aktuális felhasználó jegyeinek lekérésére szolgáló végpont. A kérés nem tartalmaz paramétereket, mivel az aktuális felhasználó jegyeit kérdezi le. A válaszban szereplő jegyek információi tartalmazzák a jegyek típusát, érvényességüket és egyéb fontos adatokat, a 6.4 ábrán látható módon.

Body Cookies (1) Headers (21) Test Results | 200 OK • 1.56 s • 2.22 KB | Save Response

{ } JSON Preview Visualize

Root / 0

ticket_uid	TIC_4DCMG
date_of_purchase	2025-03-26T18:57:43.414+02:00
expiration_date	2025-04-26T19:57:43.414+03:00
from_station_uid	STT_9CVLG
to_station_uid	STT_MBGDP
from_station_name	Gyergyó Állomás
to_station_name	Csomafalva Szászfalu
is_valid	true
route_uid	ROU_28JLW
route_name	Szászfalu-Gyergyó
ticket_price	1100.0

6.4. ábra. A GET /v1/tickets_of_user végpont kérésének paraméterei és válasza

- **POST /v1/admin/add_station_to_route** – Ez az adminisztrátorok számára elérhető végpont lehetővé teszi egy állomás hozzárendelését egy adott útvonalhoz. A kérés tartalmazza az útvonal

és az állomás azonosítóját, és a válasz megerősíti a sikeres műveletet, amint a 6.5 ábrán is látható.

Query Params			
☒	Key	Value	Description Bulk Edit
☒	license_plate	HR12VTT	
☒	route_uid	ROU_I0BF3	
	Key	Value	Description

Body Cookies (1) Headers (21) Test Results 200 OK • 1.12 s • 1.68 KB Save Response

{ } JSON Preview Visualize

success Current route saved successfully!

6.5. ábra. A POST /v1/admin/add_station_to_route végpont kérésének paraméterei és válasza

6.2. Adatbázis kapcsolat

Az API végpontjai a PostgreSQL adatbázissal kommunikálnak. Az adatbázis tranzakciókat az ActiveRecord ORM segítségével kezeljük, amely lehetővé teszi az adatok egyszerű és hatékony lekérését, beszűrésát és frissítését.

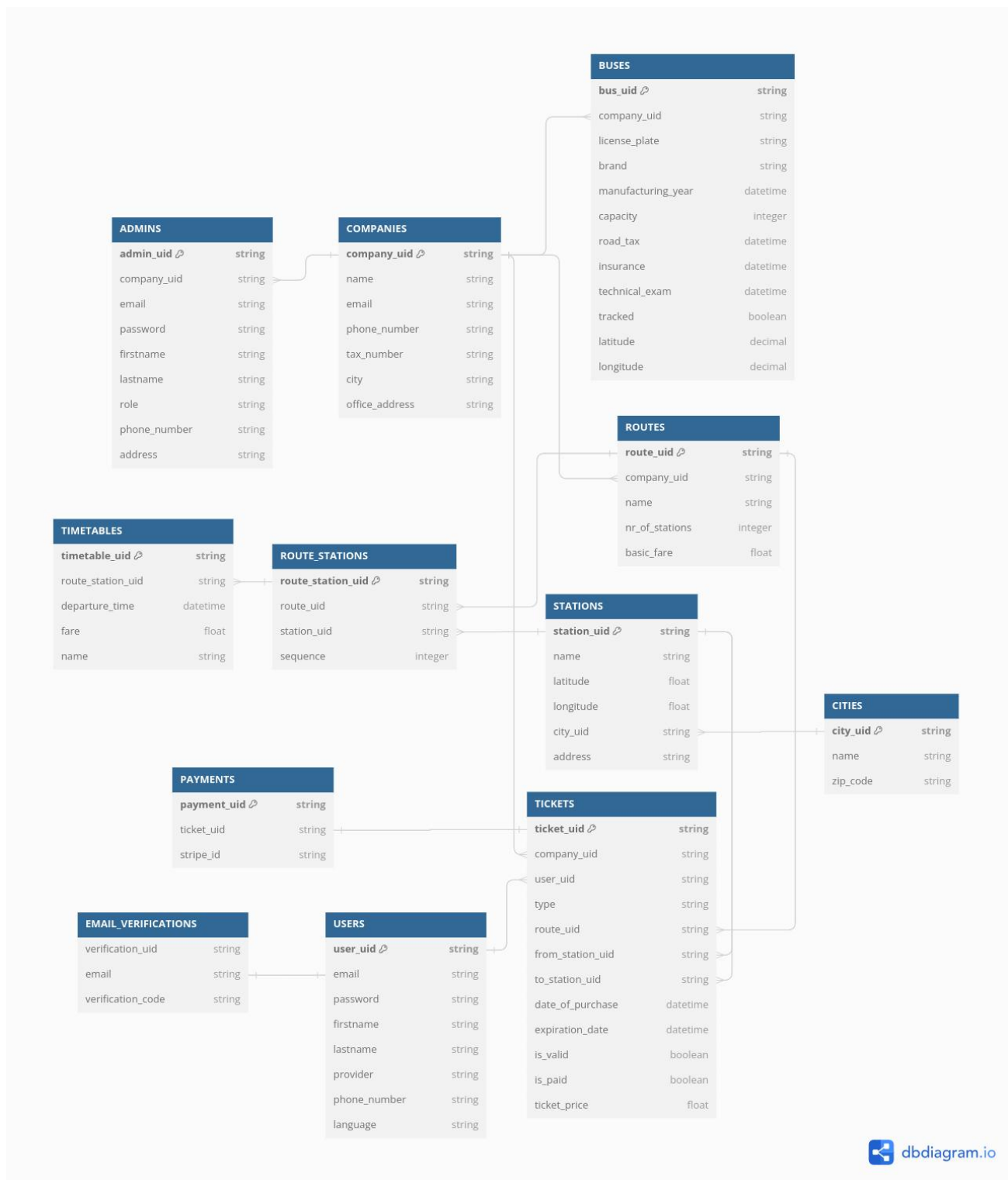
6.2.1. Adatbázis táblák

A PostgreSQL adatbázisunk a következő táblákból áll:

- **Companies:** Azon cégek adatai, akikkel szerződést kötött a Bus4U.
- **Admins:** Különböző cégekhez tartozó adminisztrátorok adatai. Szerepük lehet “boss” vagy “driver”.
- **Buses:** A cégekhez tartozó autóbuszok adatai.
- **Routes:** A cégek saját útvonalai.
- **Stations:** A rendszerben található buszmegállók összessége.
- **Route_stations:** Egy cég adott útvonalán keresztülhaladó megállók.

- **Timetables:** Egy útvonal megállójának indulási időpontjai és ára nap szerinti felosztásban.
- **Cities:** Az összes város, amelyen áthaladnak útvonalak.
- **Users:** A Bus4U felhasználói a fontosabb adataikkal.
- **Email_verifications:** Az adott email címekhez tartozó regisztrációkor felhasználandó kódok.
- **Tickets:** A felhasználókhöz tartozó jegyek.
- **Payments:** Stripe fizetések eltárolása.

A táblák közötti kapcsolatokat a 6.6 ábra szemlélteti.



6.6. ábra. Az rendszer adatbázisdiagramja

6.2.2. Objektum-relációs leképzés (ORM)

A backend API adatbázis-kezelése a Ruby on Rails keretrendszer beépített ActiveRecord objektum-relációs leképzőjével (ORM) történik. Az ORM lehetővé teszi az alkalmazás számára, hogy közvetlenül Ruby osztályok és objektumok formájában kezelje az adatbázis tábláit, így egyszerűsítve az adatbázis-műveleteket és csökkentve a fejlesztési időt.

Az ActiveRecord fő előnyei közé tartozik:

- **Egyszerű adatlekérdezés:** A komplex SQL utasítások helyett Ruby nyelven írhatók a lekérdezések.
- **Adatkapcsolatok automatikus kezelése:** Egyértelművé teszi az adatok közötti kapcsolatokat (például `belongs_to`, `has_many`, `has_one`).
- **Validációk és tranzakciók egyszerű kezelése:** Adatvalidáció, tranzakciókezelés és hibaüzenetek egyszerű integrációja az üzleti logikába.
- **Migrációk támogatása:** Könnyű adatbázis-struktúra módosítás a migrációs fájlokon keresztül.

Az ActiveRecord használata jelentősen egyszerűsíti a fejlesztési folyamatot, lehetővé téve, hogy a fejlesztők elsősorban az üzleti logikára összpontosítsanak, miközben a rendszer hatékonyan és biztonságosan kommunikál a PostgreSQL adatbázissal.

7. fejezet

A felhasználói felület tervezése

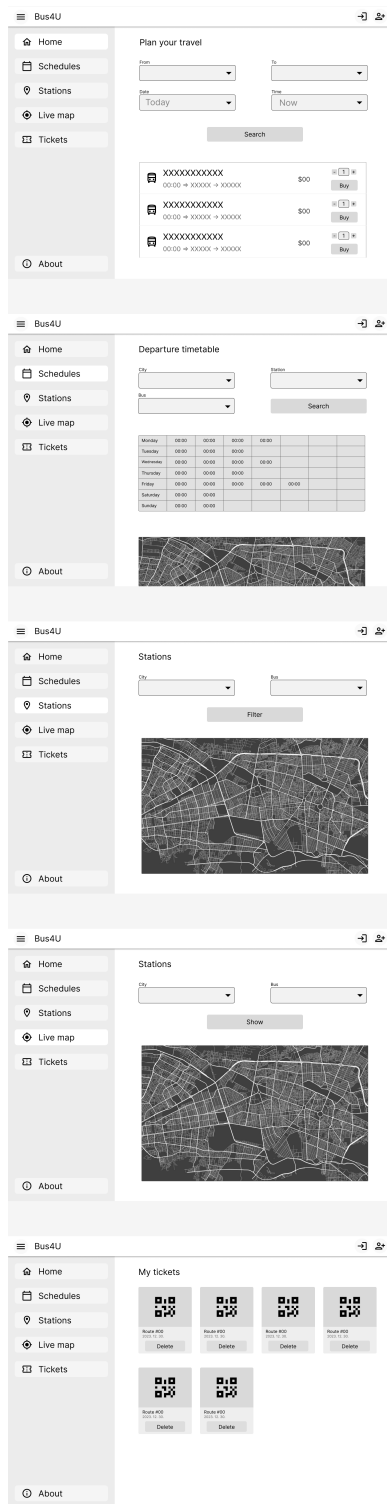
Az egyik legfontosabb része a projektnek a UI, mert ezt a részét látják a felhasználók és ezzel fognak interakcióba lépni. Emiatt kiemelkedően fontos, hogy az jól meg legyen tervezve. Szerencsére ennek a feladatnak a megkönnyítésére léteznek jól bevált módszerek, amelyek nagyban felgyorsítják és megkönnyítik a munkát. Az mi választásunk a Figma tervezőprogramra esett.

7.1. Figma terv

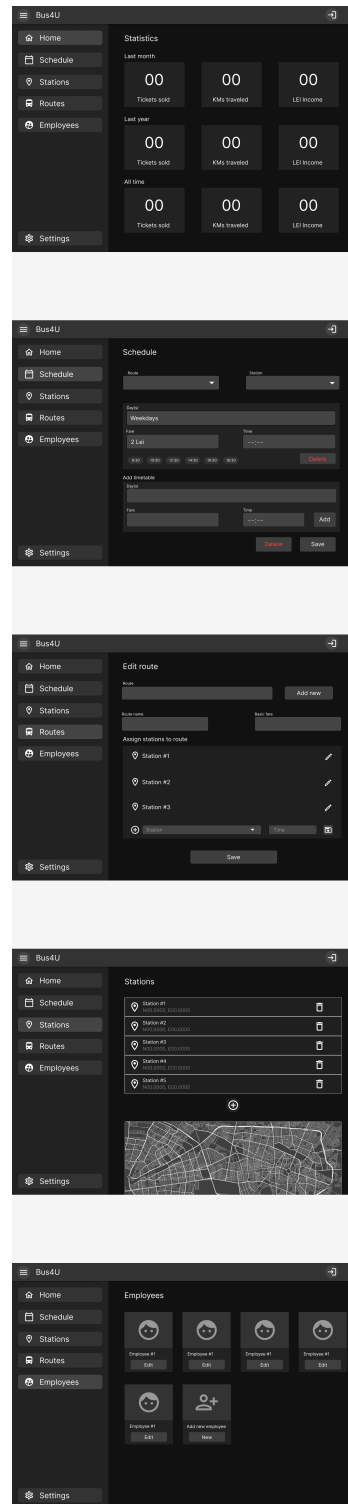
A UI terv vizuális elkészítésére a Figma¹ segédprogram a legmegfelelőbb választás rengeteg szempontból. A Figma egy olyan felülettervező program, amely lehetővé teszi több felhasználó együttműködését is, például a dizájnerek és a fejlesztők közös munkáját. A Figmában egyszerűen, fogd és vidd módszerrel lehet hozzáadni elemeket a munkavászonhoz, de aki pontosabban szeretne tervezni, annak az oldalsávban van lehetősége pixel pontossággal megadni az elemek tulajdonságait is. A Figmában elérhetőek előre elkészített felületi elemek is, amelyek használata sokszorosan lerövidíti a tervezéshez szükséges időt. A felületi elemekhez funkcionalitást is lehet rendelni, például ha egy gombra kattint a felhasználó, akkor egy másik oldalra navigáljon.

A Figma-ban elkészített terveket követve sokkal gyorsabban lehetett fejleszteni a felhasználói felületet. Az általunk követett felhasználói weboldal terve a 7.1 ábrán, az adminfelület terve a 7.2 ábrán, míg a mobilalkalmazás terve a 7.3 ábrán látható.

¹Figma: <https://www.figma.com>



7.1. ábra. A felhasználói weboldal terve



7.2. ábra. Az admin weboldal terve



Bus4U

Log In

Email

Password

Log In

Not a member yet?

Register

 Bus4U

Register

Email

Password

Confirm password

Firstname

Lastname

Register

Already registered?

Login

8. fejezet

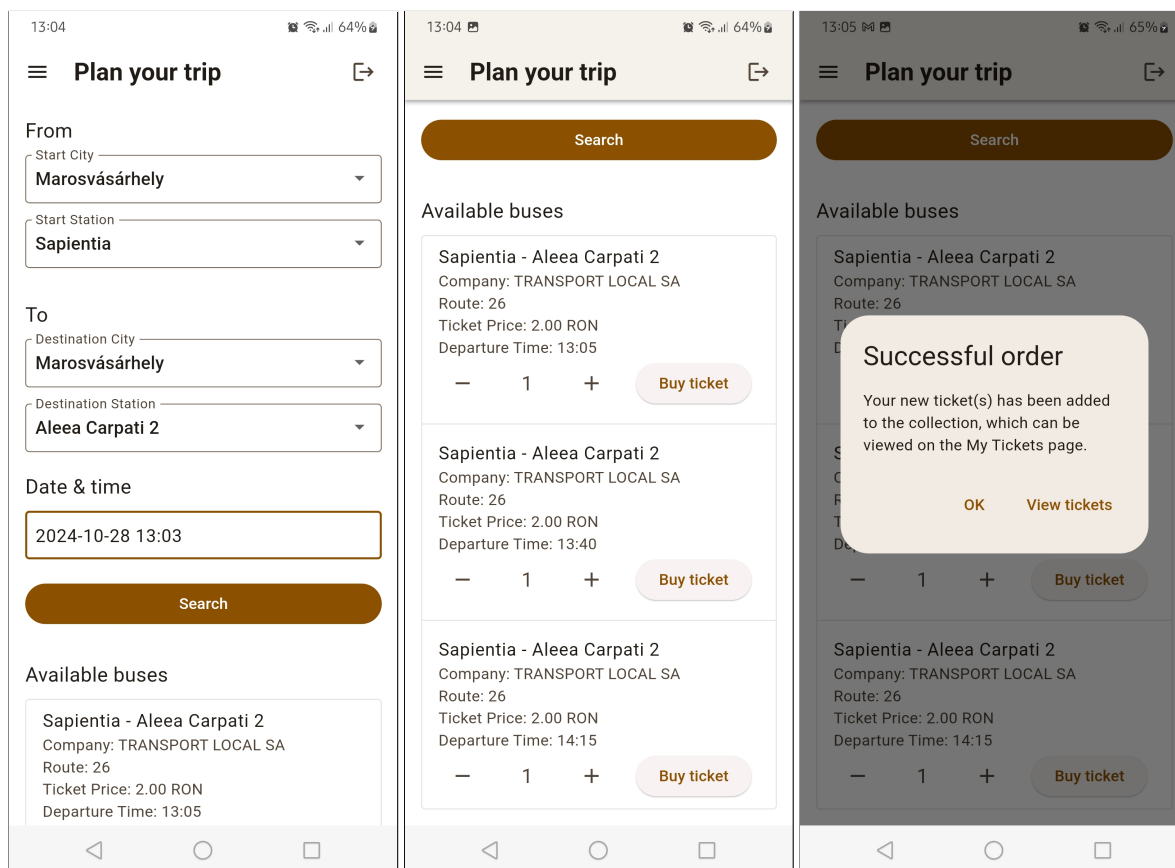
Gyakorlati megvalósítás

8.1. Felhasználói weboldal és alkalmazás

A felhasználói felület működése megegyezik a weboldalon és a mobilalkalmazásban is, azonban a megjelenésük kicsit eltér egymástól, mivel a weboldal a böngészőben fut, míg az alkalmazás a mobil eszközön. Ettől függetlenül a weboldal is tökéletesen használható az okostelefonokon, mivel alkalmazkodik bármilyen kijelzőmérethez, így aki nem szeretne alkalmazást telepíteni, az is könnyen hozzáférhet a Bus4U-hoz böngészőből. A felületek törekednek a hasonlóságra, így a navigációs menü, a UI komponensek és a színpaletta is megegyezik, ezáltal a felhasználók könnyen átláthatják és használhatják mindkét platformot.

8.1.1. Főoldal

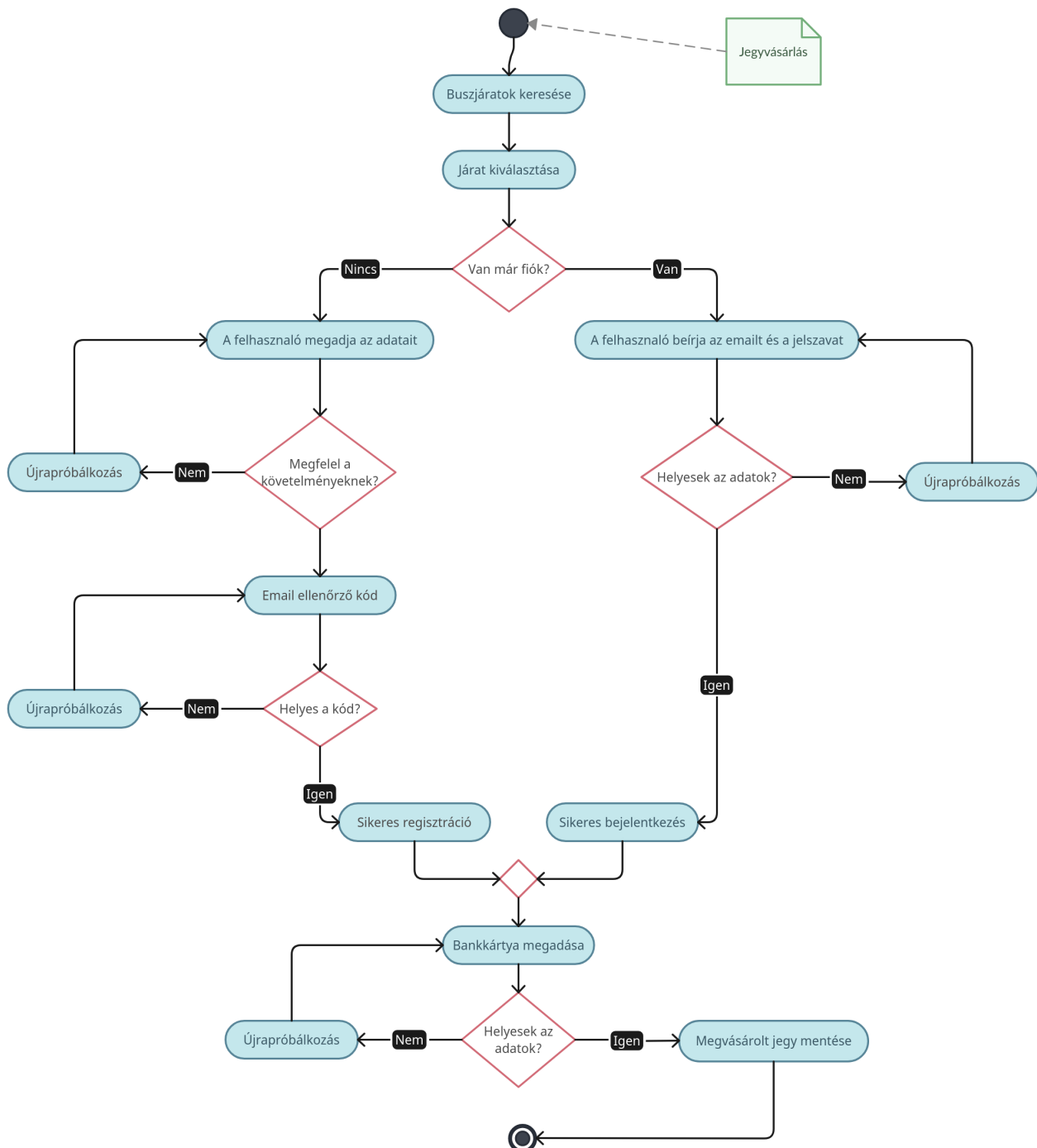
A weboldalra rákeresve a kezdőoldal fogad, ahol lehetőség van egy utazás megtervezésére. Ehhez ki kell választani a kiindulási helyet és megállót, majd a cél helyét és a megállóját, végül pedig az utazás dátumát és időpontját. Ekkor a rendszer lekéri a szerverről a megadott megállók közötti, az adott napon és a megadott időpont után közlekedő járatokat, valamint azok jegyárát. Minden egyes megjelenő járatnál ki lehet választani a jegyek számát, amely alapértelmezetten egy jegyre van beállítva. Ezt követően a felhasználó, a vásárlás gombra kattintva megszerezheti a jegy(ei)t az adott buszjáratra, amennyiben be van jelentkezve, ellenkező esetben a rendszer átirányítja a bejelentkezéshez.



8.1. ábra. Jegyvásárlás a mobilalkalmazásban

A jegyvásárlás folyamata a 8.1 ábrán látható. A könnyebb megértés végett, a jegyvásárlási és bejelentkezési folyamatot szemlélteti a 8.2 ábrán lévő aktivitás diagram. Ahogy ott látható, miután a felhasználó megtalálta a megfelelő buszjáratot és rálépett a jegy megvásárlására, a rendszer átirányítja őt a bejelentkezéshez. Ha van már fiókja, bejelentkezhet a meglévő adataival, amennyiben nincs, úgy átléphet a regisztrációs oldalra, ahol új fiókot hozhat létre. A regisztráció során a rendszer minden adatot ellenőriz és ha valamelyik hibás (pl. érvénytelen email cím vagy túl rövid jelszó) akkor egy ennek megfelelő hibaüzenetet ír ki. Ez addig ismétlődik, amíg minden adat megfelelő lesz, aztán a szerver küld egy 4 számjegyből álló ellenőrző kódot a megadott e-mail címre, amelyet a felhasználó be kell írjon az alkalmazásban megjelenő szövegmezőbe. Ha ez a kód helytelen, akkor a folyamat kezdődik előlről. Amennyiben a regisztráció sikeres és az ellenőrzőkód helyes, akkor a rendszer átirányítja a felhasználót a bankkártyás fizetési oldalra, ami jelenleg nincs implementálva, ez a jövőbeli tervek

között szerepel. Végül a backend generál egy egyedi kódot a megvásárolt jegyhez és megjelenik a kijelzőn a sikeres vásárlás visszaigazolása.

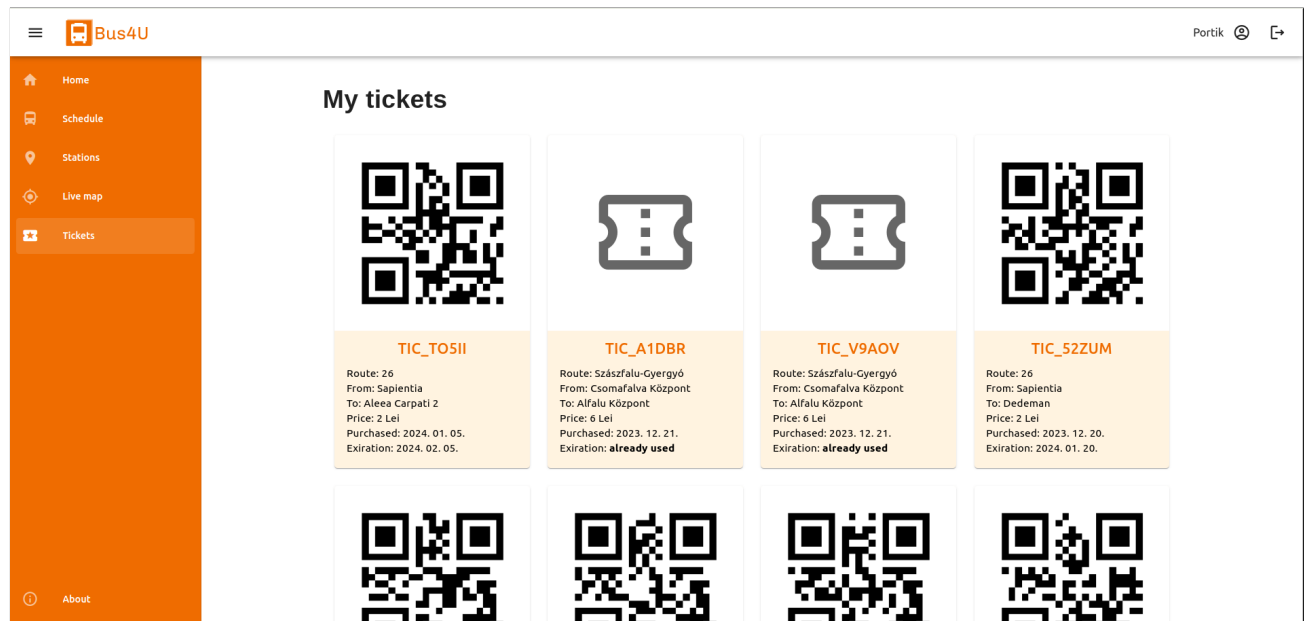


8.2. ábra. A jegyvásárlás folyamata

8.1.2. Jegyek

A megvásárolt jegyeket a rendszer elmenti a felhasználó gyűjteményébe, amely a Jegyek oldalon található, így ezek bármikor visszakereshetők és felhasználhatóak lesznek, kivéve ha lejár az egy hónapos érvényesség. A jegyen megjelenített adatok között van a járat neve, a kiinduló- és célállomás, az ár, valamint a vásárlás és az érvényesség dátumai. A már elhasznált jegyek megvannak jelölve és el van rejtve a rajtuk lévő QR-kód így azokat nem lehet többször felhasználni. A jegyek a létrehozási idejük szerinti csökkenő sorrendben vannak listázva, így a legfrissebb jegyek mindig az elején vannak. A webfelületen listázott jegyek a 8.3 ábrán láthatóak.

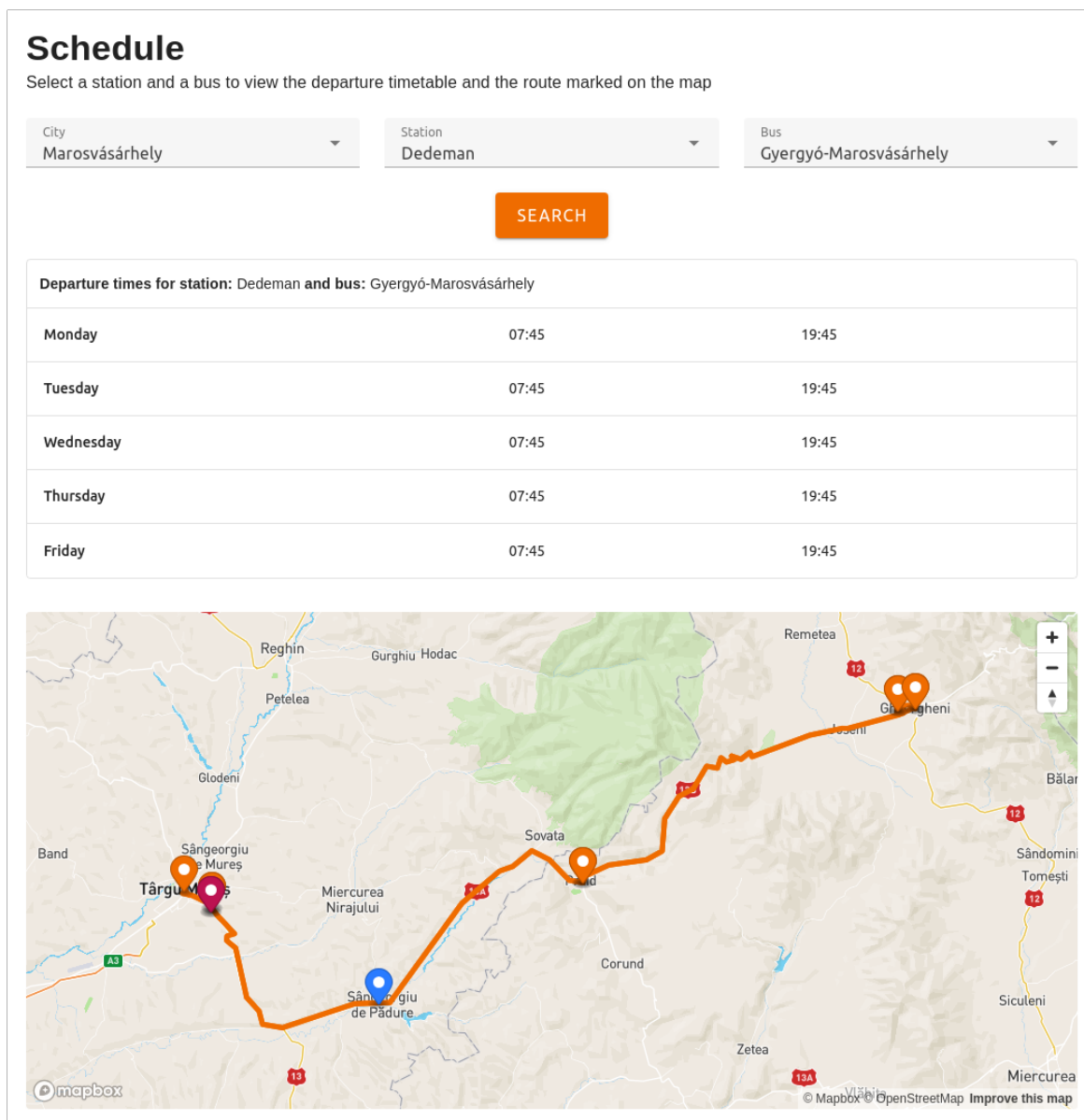
A itt megjelenő jegyeket úgy lehet felhasználni, hogy a buszra való felszálláskor a sofőr mellett elhelyezett POS terminálra kell mutatni az aktuális jegy QR-kódját, az pedig beolvassa és elküldi a szervernek ellenőrzésre. Ha a kód érvényes és minden rendben van, akkor a felhasználó felszállhat a buszra, a jegy pedig érvénytelenítődik.



8.3. ábra. A felhasználó megvásárolt jegyei

8.1.3. Menetrend

A következő oldal a Menetrend, amely hasonlóan az egyes buszmegállókban kihelyezett táblázatokhoz, megjeleníti a kiválasztott buszjárat indulási időpontjait, az adott megállóból, napokra lebontva. A mi megoldásunk viszont tartalmaz egy interaktív térképet is, ahol a felhasználó meg tudja nézni a kiválasztott járat útvonalát, az érintett megállókat és az útvonalon közlekedő buszok élő helyzetét is. Egy példa a menetrendre és a térképen megjelenő útvonalra a 8.4 ábrán látható.



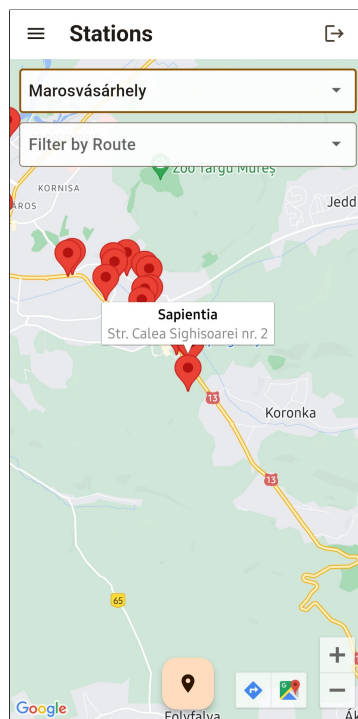
8.4. ábra. Listázott menetrend és az útvonal térképe a webfelületen

8.1.4. Megállók és élő térkép

Ez a két oldal az előbb említett térképes funkciókat mutatják meg külön-külön, egyszerűbben. A Megállók oldal megjeleníti a térképen az összes megállót, és ezek között szűrést is biztosít város vagy buszjárat szerint. Az egyes megállókra kattintva, a térképen megjelenik egy kis buborék a kiválasztott megálló adataival, mint a név és a cím, így könnyebb megtalálni a keresett megállót. A térképen megjelenő megállóról példa a 8.5 ábrán látható.

Az élő térkép oldalon pedig egy-egy útvonalon közlekedő buszok élő pozícióját lehet követni, miután kiválasztottuk a várost és az útvonalat. Ezeket az adatokat a buszokon lévő POS terminálok szolgáltatják, így biztosak lehetünk abban, hogy a buszok helyzetei valós időben jelennek meg. Ez a funkció abban segít, hogy ne maradjunk le, ha egy busz siet, illetve ne várakozzunk feleslegesen a megállóban, ha egy busz késik, így időt lehet vele spórolni.

Mindkét oldalon megjelenik a felhasználó jelenlegi pozíciója egy kék gombostű formájában, ha engedélyezte az ehhez való hozzáférést a telefonján, ezáltal könnyebben meg lehet találni a legközelebbi megállót vagy buszt.



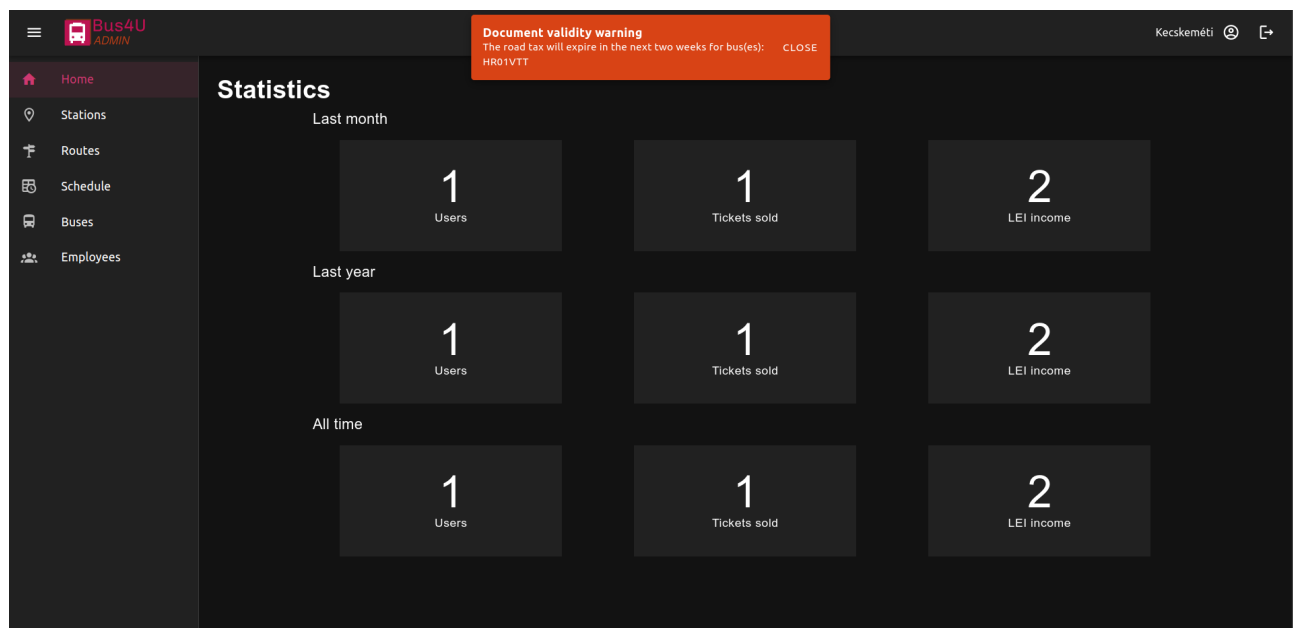
8.5. ábra. Megállók

8.2. Adminisztrátori weboldal

Az adminisztrátori felület csak bejelentkezéssel elérhető, ehhez pedig egy admin típusú felhasználói fiók szükséges, amely a kezelendő céghez van hozzárendelve. A cégek adatai el vannak különítve, így minden adminisztrátor csak a saját cégéhez tartozó adatokat láthatja és módosíthatja. Egy cégnek végtelen számú admin fiókja lehet, viszont ezek között lehetnek különbségek, amelyet a szerepkörök határoznak meg. Egyes adminok csak bizonyos funkciókat érhetnek el, míg mások mindenhez hozzáférhetnek. A sofőr szerepkörben lévő felhasználó például csak a buszok adatait láthatja és kezelheti, míg a menedzser szerepkörű adminisztrátor minden egyéb funkcióhoz is hozzáfér.

8.2.1. Kezdőoldal

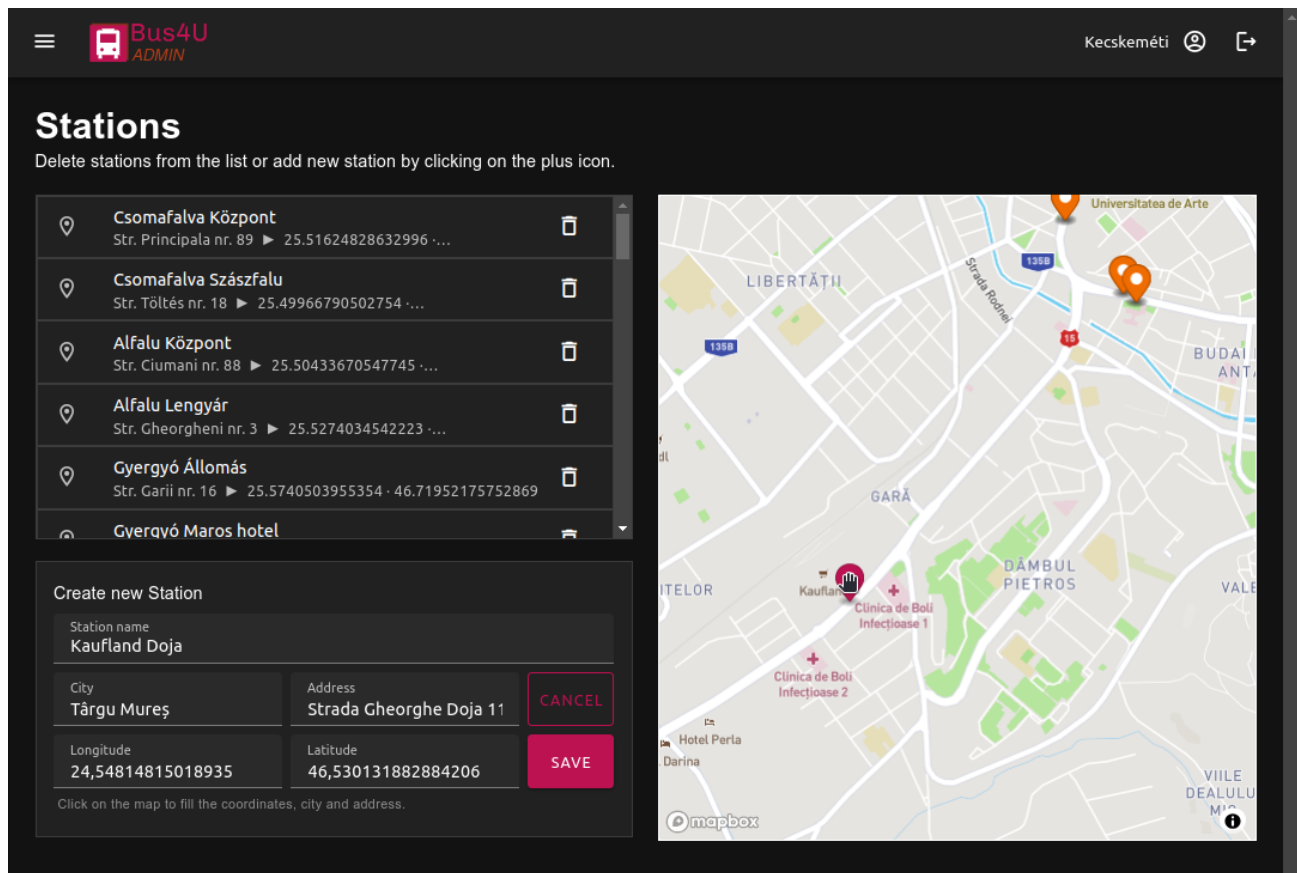
Sikeres bejelentkezés után a kezdőoldal jelenik meg, amelyen látható egy kis statisztika a regisztrált felhasználók számával, a megvásárolt jegyek számával és a bevétellel, mindezek havi, évi és mindenkor bontásban. Ezek segíthetnek a cégvezetőknek a tanulságok levonásában és a fontos döntések meghozatalában is. Ezen kívül itt jelennek meg az esetleges figyelmeztetések és értesítések is, mint például a buszok lejárt útdíja vagy a közelgő műszaki vizsgálat. A kezdőoldal a 8.6 ábrán látható.



8.6. ábra. Kezdőoldal statisztikával és értesítéssel

8.2.2. Megállók

Ezen az oldalon egy lista fogad a létező megállókkal és ezek adataival, mint a név, a hely és a földrajzi koordináták. Egy megállóra kattintva, megjelenik annak pontos helyzete a térképen. A lista elemei törölhetőek, de az oldal alján létre lehet hozni új megállókat is. Ehhez csak meg kell adni a megálló nevét, a címet és a koordinátákat. A rendszer képes automatikusan is kitölteni majdnem az összes adatot, ha a felhasználó kijelöl egy helyet a térképen, ahogy ez 8.7 ábrán is látszik.

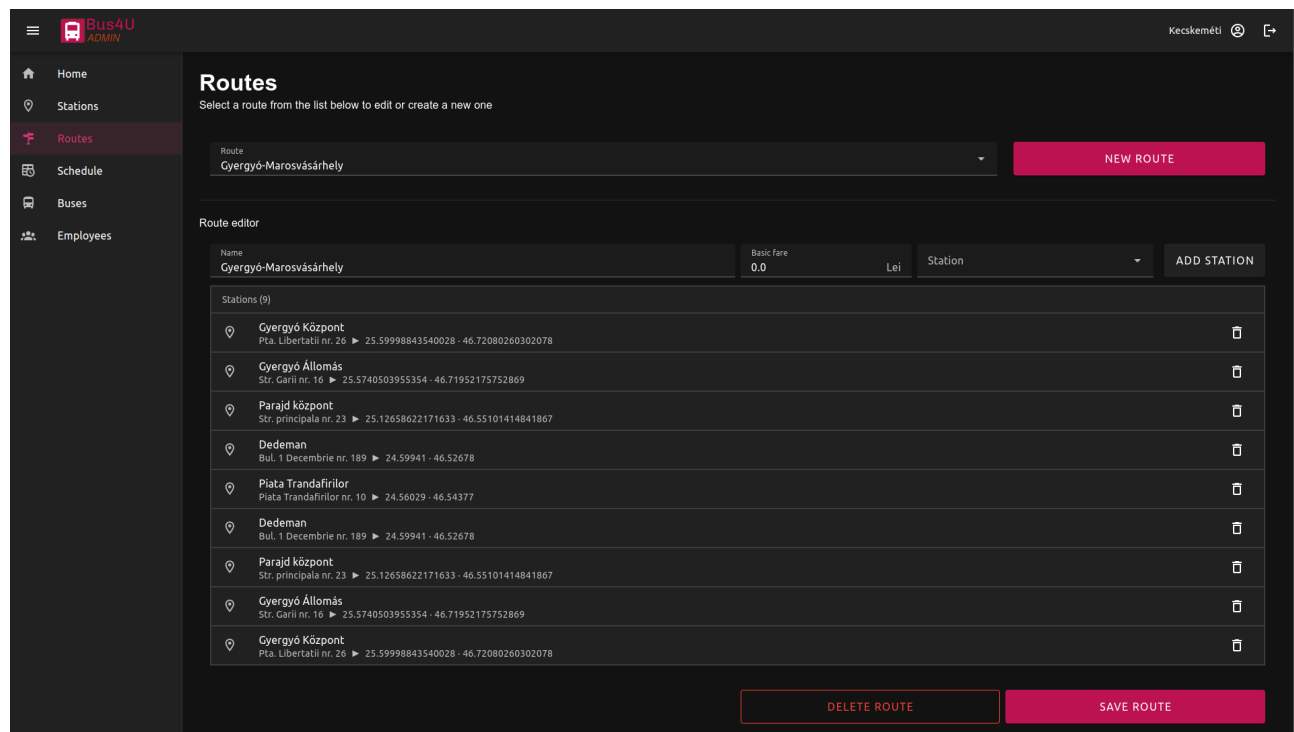


8.7. ábra. Adminisztrátori megállók oldal

8.2.3. Útvonalak

Az Útvonalak oldal a járatok útvonalainak szerkesztésére szolgál. Kezdsnek ki kell választani egy létező járatot, de akár újat is lehet létrehozni. Ezután megjelenik az útvonal-szerkesztő, ahol meg kell adni a nevet, az alap jegyárat és ki kell választani az érintett megállókat. Alább megjelenik az eddig

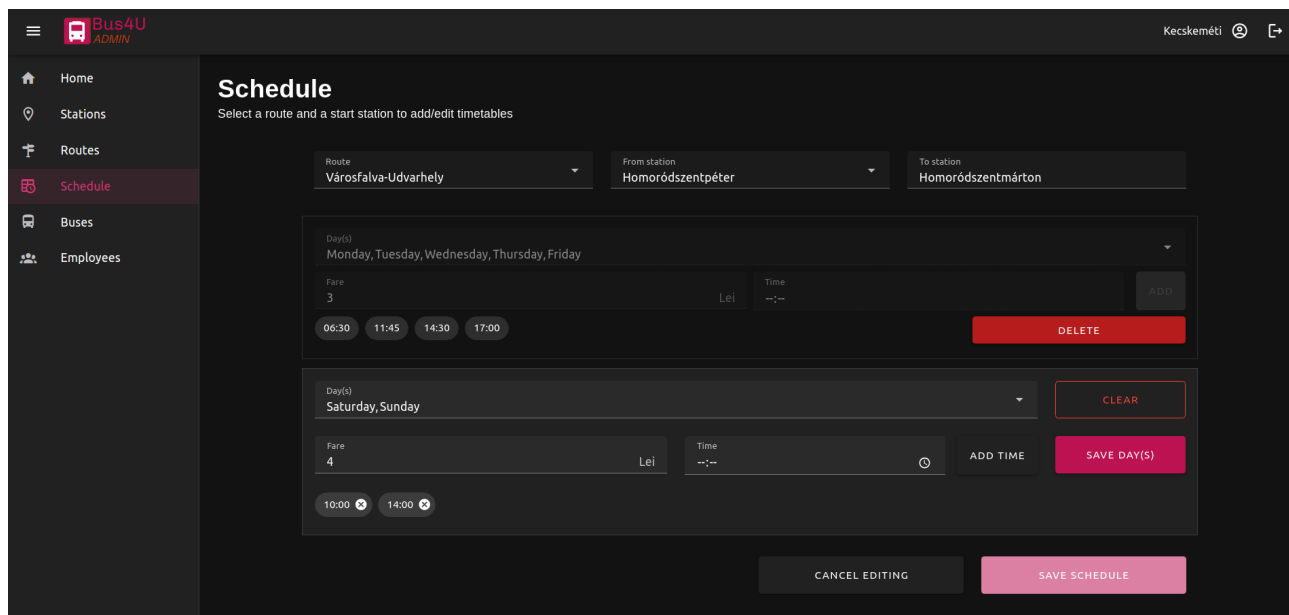
hozzáadott megállók listája, az oldal legalján pedig el lehet menteni a kiválasztott útvonalat vagy akár ki is lehet törölni, ha már nem használják. Az útvonal-szerkesztő, benne egy betöltött útvonallal a mellékelt 8.8 ábrán látható.



8.8. ábra. Útvonalak létrehozása, módosítása és törlése

8.2.4. Menetrend

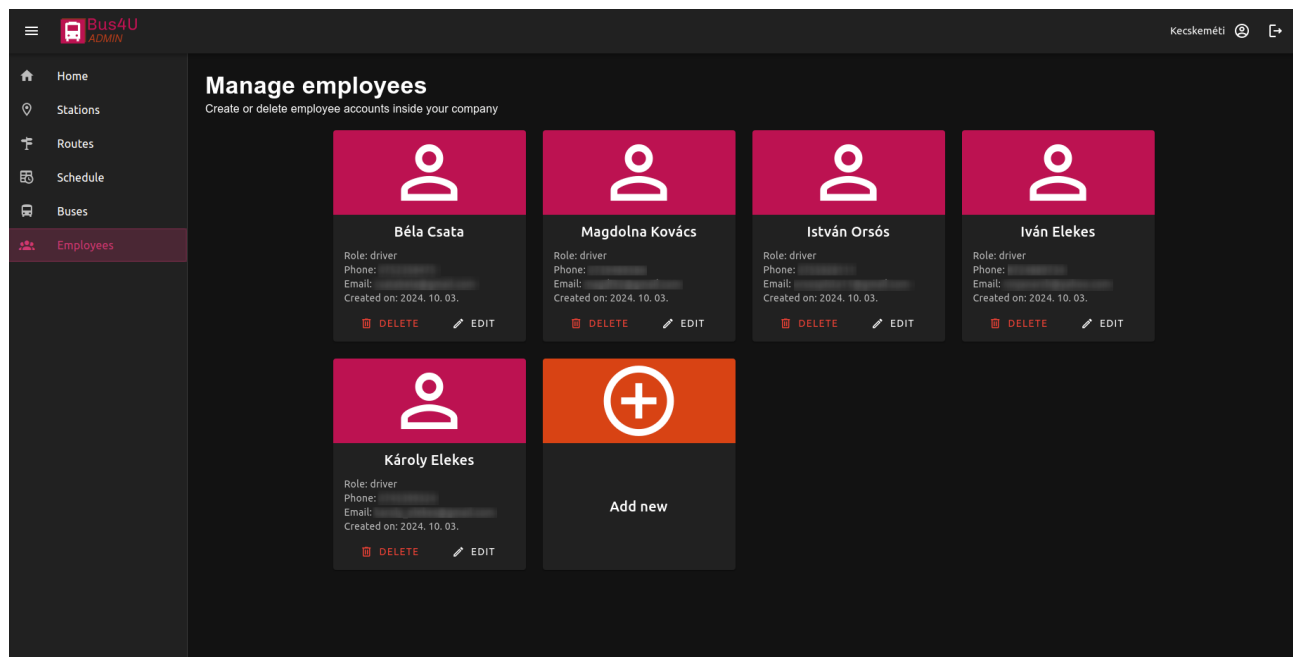
A következő oldal a Menetrend, ahol a létrehozott útvonalak megállóihoz lehet órarendeket hozzárendelni. Itt az útvonal és megálló kiválasztása után meg kell adni az órarendre érvényes napokat, a következő megállóig megszabott jegyárat és az indulási időpontokat. Egy megállóhoz hozzá lehet adni több ilyen órarendet is, így el lehet különíteni a hétvégét a hétköznapiaktól, de akár minden napra lehet más-más órarendet is megadni, ha különböznek az indulási időpontok vagy akár a jegyárak. Az órarendeket egyesével lehet szerkeszteni és törölni is, csak az a fontos, hogy a módosításokat a felhasználó az oldal alján található gombbal mentse el, különben nem lesznek érvényesek. A 8.9 ábrán látható egy példa a menetrend szerkesztésére.



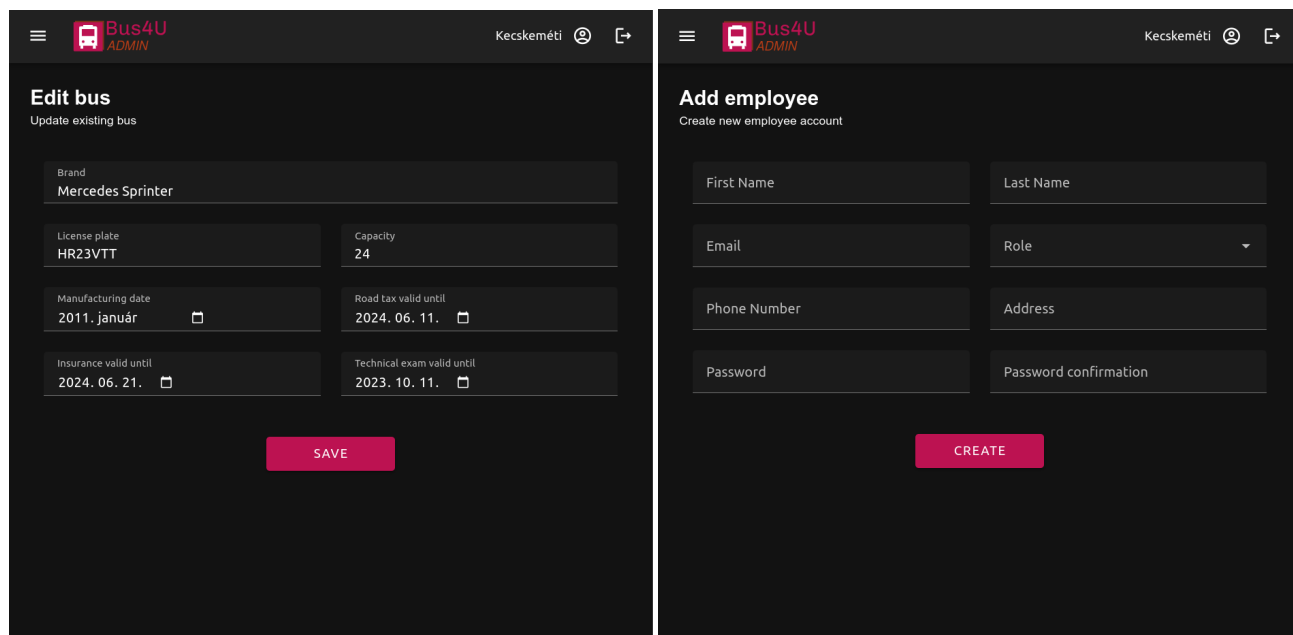
8.9. ábra. Menetrend szerkesztése

8.2.5. Buszok és alkalmazottak

Az utolsó két oldalon a buszokat és az alkalmazottakat lehet kezelni. Ez a két oldal hasonlóan néz ki és hasonlóan működik. Mindkét esetben látható egy lista a meglévő buszokkal vagy alkalmazottakkal, ahol minden elemet egy ú.n. kártya komponens reprezentál. A kártyák tartalmazzák az elemek összes tulajdonságát, és két gombot, amelyekkel lehet módosítani vagy törölni őket. Az utolsó kártya segítségével újabb példányt is lehet létrehozni az adott kategóriában. A buszok létrehozásánál meg kell adni a márkát, a rendszámot, a férőhelyek számát, a gyártási évet, valamint az útdó, a biztosítás és a műszaki engedély lejárat dátumát. Az utóbbiakról később emlékeztető fog megjelenni a weboldalon, amikor közeledik azok lejárat ideje. Új alkalmazott hozzáadásánál meg kell adni a nevet, e-mail címet, beosztást (ez adja majd meg a jogait a weboldalon), telefonszámot, lakcímet (ez opcionális), és egy új jelszavat. Az alkalmazottak listája és a rajta szereplő kártyák a 8.10 ábrán láthatóak, míg a busz szerkesztésére és az alkalmazott létrehozására példa a 8.11 ábrán.



8.10. ábra. Az alkalmazottak listája

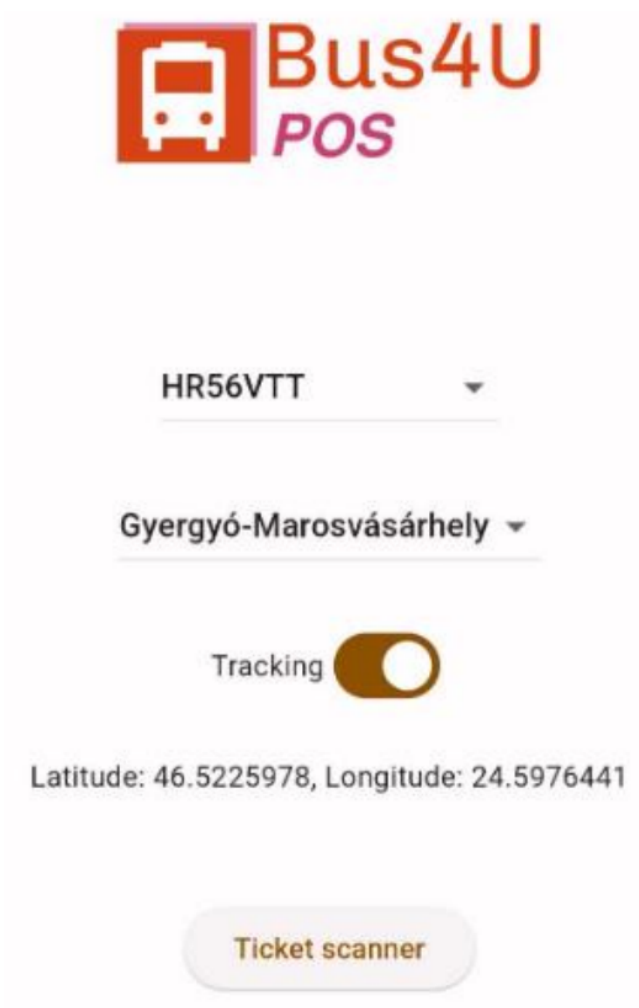


8.11. ábra. Busz szerkesztése és alkalmazott létrehozása

8.3. POS terminál alkalmazása

8.3.1. GPS oldal

Miután a sofőr bejelentkezett, ezen az oldalon lehetőség nyílik kiválasztani az aktuális buszt, amelyiket jelenleg a sofőr vezet, illetve azt az útvonalat, amelyet éppenséggel végrehajt. Található egy kapcsoló, amellyel el lehet indítani az autóbusz lokációjának követését. Ez félpercenként küldi fel az API-ra a busz lokációját, ezáltal lehet a weboldalon és az alkalmazásban élőben követni a busz helyzetét. Legalul egy Ticket scanner gomb fedezhető fel, amely átirányít a jegyszkennelő oldalra. A főmenü elrendezése a 8.12 ábrán látható.



8.12. ábra. Busz kiválasztása és GPS követés

8.3.2. Jegyszkennelő oldal

Ez az oldal a kamerát jeleníti meg. Itt kell beszkenyelni a jegyet. Szkennelés után egy felugró ablak jelenik meg 5 másodpercig, három üzenetet hordozhat: Ha sikeresen érvényesítve van a jegy, akkor zöld háttérrel kiírja hogy sikeres érvényesítés és jó utazást kíván az alkalmazás. Ha a jegy már érvényesítve volt, vagy érvénytelen a QR kód, ezt egy piros hibaüzenettel jelzi. Illetőleg ha az API valamilyen oknál fogva leáll, akkor ennek a hibájáról is üzenetet kapunk. A lehetséges üzenetek tartalma a 8.13 ábrán látható.



8.13. ábra. Jegy szkennelése

9. fejezet

Összefoglalás

Befejezésképpen kijelenthető, hogy a projekt sikeres, mivel a leszögezett terveket sikerült implementálni, sőt fejlesztés közben egyéb hasznos funkcionalitásokat is sikerült hozzáadni a végső eredményhez. Ezáltal a végfelhasználóknak egy-egy funkciódús és jól működő weboldal és applikáció áll a rendelkezésükre, amely nagyban megkönnyíti a tömegközlekedésben való részvételt, bármilyen eszközön is próbálnák elérni ezt. A felhasználók pillanatok alatt tervezhetik meg az utazásaikat, vásárolhatnak jegyet, informálódhatnak megállókról, útvonalakról és órarendekről, sőt a buszok élő pozíciót is nyomon követhetik a térképen. A busztársaságoknak pedig egy egyszerű vezérlőfelület van létrehozva, amely segítségével könnyen és gyorsan tudják kezelni a buszokat, az útvonalakat, a menetrendeket, a jegyárakat és az alkalmazottakat, az üzleti döntéseik megkönnyítésére pedig egy statisztika is a rendelkezésükre áll.

Következtetésképpen a Bus4U egy reális piaci rést tud betölteni, mivel megoldja a tömegközlekedéssel kapcsolatos problémákat és ezáltal vonzóbbá teszi az ilyen eszközök használatát az emberek számára. Ez azért fontos, mert a buszokkal való közlekedés elengedhetetlen a forgalom, és a környezetszennyezés csökkentéséhez, de jelenleg, a rendszer hiányosságai miatt ez egy nem túl népszerű közlekedési eszköz hazánkban.

9.1. További fejlesztési lehetőségek

Felhasználók szempontjából a következő fontos fejlesztés a buszok telítettségének valós idejű követése lenne, mert ezzel elkerülhető lenne a zsúfoltság a buszokon, és így sokat javulna a tömegközlekedés minősége. Be szeretnénk idővel vezetni a bérletek, illetve különböző kedvezmények kezelésének lehetőségét is, mert ez is növelheti a népszerűséget. Hasznos lenne az útvonaltervezés kibővítése gyalogos navigációval, valamint a távolság, idő és forgalom szempontjából legoptimálisabb útvonal felajánlásával, így az alkalmazás el tudná vezetni a felhasználót a számára legmegfelelőbb megállóig. A különböző értesítések kiküldése is hasznos lehet, például ha egy busz késik, vagy ha egy útvonalon változás történik, fontos lenne, hogy időben informálódjon erről a felhasználó.

A kényelmi funkciók bővítése mellett, be lehetne vezetni egy kis játékosítást is, például a buszokon való utazásokért járó pontokat lehetne összegyűjteni, amelyeket később be lehetne váltani kedvezményekre, vagy elérhetővé tenni különböző mérföldköveket, amelyeket a felhasználók elérhetnek és gyűjthetnek. A pontok mellett a kilométereket is lehetne gyűjteni, amelyek megjelennének a felhasználók profiljaiban, és így versenyezhetnének egymással. Ezek a játékos megoldások egy kis színt vihetnek az unalmas buszozásba, így nagyban növelhetik a felhasználók aktivitását és hűségét.

A busztársaságok szempontjából is rengeteg lehetőség áll rendelkezésre. Mivel a projektet úgy terveztük, hogy több busztársaság is fel tudja használni, így ez jelen pillanatban általános használatra van optimalizálva. Ennek ellenére egy-egy partner cég kérésére személyre is szabhatnánk a rendszert, egyedi funkciókat és kinézetet bevezetve. Az is hasznos lenne például, ha a cég akár a teljes könyvelését le tudná itt bonyolítani: az alkalmazottak munkaóráit automatikusan vezetni, az autóbuszok tankolásait összesíteni, illetve buszokon lévő POS terminál GPS moduljának segítségével akár a "Fo-aie de parcurs" vezetése is automatizálható lenne. A könyveléshez a járművek szervízköltségeinek számláit is kellene gyűjteni az oldalra, amivel további hasznos kimutatásokat tudnánk generálni.

Ezek jutottak eddig eszünkbe, de még rengeteg lehetőség van olyan fejlesztésekre, amelyek növelhetik a Bus4U hatékonyságát és népszerűségét mind a felhasználók, mind a busztársaságok számára.

Irodalomjegyzék

- [1] M. Cropper and P. Suri, „Measuring the air pollution benefits of public transport projects,” *Regional Science and Urban Economics*, vol. 107, p. 103976, 2024.
- [2] T. P. L. Le and T. A. Trinh, „Encouraging public transport use to reduce traffic congestion and air pollutant: A case study of ho chi minh city, vietnam,” *Procedia engineering*, vol. 142, pp. 236–243, 2016.
- [3] M. Orošnjak, M. Jocanović, B. Gvozdenac-Urošević, D. Šević, L. Duđak, and V. Karanović, „Bus fleet management—a systematic literature review,” *Promet-Traffic&Transportation*, vol. 32, no. 6, pp. 761–772, 2020.
- [4] P. Polyviou, *Modelling traffic incidents to support dynamic bus fleets management for sustainable transport*. PhD thesis, University of Southampton, 2011.
- [5] O. Elijah, S. L. Keoh, S. K. bin Abdul Rahim, C. K. Seow, Q. Cao, M. A. bin Sarijari, N. F. Ibrahim, and A. Basuki, „Transforming urban mobility with internet of things: public bus fleet tracking using proximity-based bluetooth beacons,” *Frontiers in the Internet of Things*, vol. 2, p. 1255995, 2023.
- [6] N. Hounsell, B. Shrestha, and A. Wong, „Data management and applications in a world-leading bus fleet,” *Transportation Research Part C: Emerging Technologies*, vol. 22, pp. 76–87, 2012.
- [7] V. ROLLAND and F. GÁBOR, „Infrastrukturális vagy közösségi érzékelés az okos városokban?,” *Híradástechnika*, vol. 71, no. 1, pp. 47–51, 2016.
- [8] E. Hanchett and B. Listwon, *Vue.js in Action*. Simon and Schuster, 2018.

- [9] P. D. Garaguso, *Vue.js 3 Design Patterns and Best Practices: Develop scalable and robust applications with Vite, Pinia, and Vue Router*. Packt Publishing Ltd, 2023.
- [10] E. Windmill, *Flutter in action*. Simon and Schuster, 2020.
- [11] M. Hartl, *Ruby on Rails Tutorial: Learn Web Development with Rails*. Addison-Wesley Professional, 2016.
- [12] O. Fernandez, *The Rails Way*. Addison-Wesley Professional, 2017.
- [13] K. Douglas and S. Douglas, *PostgreSQL: Up and Running*. O'Reilly Media, 2015.
- [14] J. Murrell, *PostgreSQL 12 High Availability Cookbook*. Packt Publishing, 2019.
- [15] B. Wilder, *Cloud architecture patterns: using Microsoft Azure*. O'Reilly Media, Inc., 2012.
- [16] M. Collier and R. Shahan, *Microsoft Azure Essentials-Fundamentals of Azure*. Microsoft Press, 2015.